

UNIVERSIDAD FRANCISCO DE VITORIA

ESCUELA POLITÉCNICA SUPERIOR

COMPUTER SCIENCE DEGREE

FINAL DEGREE PROJECT

RESEARCH TRACK

Word-Level American Sign Language Recognition from Hand Landmarks

Francisco Sandoval Hernández

Tutor: Moisés Martínez Muñóz

CALIFICACIÓN DEL PROYECTO DE FIN DE GRADO

CUALITATIVA:	
NUMÉRICA:	

Conforme Presidente:	**Conforme Secretario:**
Fdo.:	Fdo.:

Conforme Vocal:	**Conforme Vocal:**	**Conforme Vocal:**
Fdo.:	Fdo.:	Fdo.:

Lugar y fecha: Pozuelo de Alarcón, a ____ de _____ de 2026

Acknowledgements

I would like to thank my supervisor, Moisés Martínez Muñóz, for the technical guidance, the steady feedback, and the willingness to push back on every shortcut along the way. The shape of this project owes a great deal to those conversations.

I am also grateful to the Escuela Politécnica Superior of Universidad Francisco de Vitoria for providing the academic environment in which this project took shape, and to the lecturers whose courses laid the foundations on which the rest was built.

Finally, I want to thank my family and friends for their patience and support throughout the long stretches of work that this thesis required.

Abstract

Sign language recognition serves to mitigate communication barriers between deaf or hard-of-hearing signers and non-signing people. This Final Degree Project develops and evaluates a research prototype for word-level American Sign Language (ASL) recognition from short video clips. The work first reviews the accessibility problem, the computer vision methods used for recognition and the technical requirements of the system. It then follows the methodological evolution of the project, from initial approaches based on hand detection and static letter recognition towards a sequence-based model that represents the movement of signs across frames. The final system uses MediaPipe Hands to extract 21 3D hand landmarks per frame, normalises the resulting sequences and classifies each clip with a bidirectional Gated Recurrent Unit (GRU) with a temporal attention mechanism. The experiments evaluate architecture choices, sequence length, cross-dataset evaluation and vocabulary size using the Word-Level American Sign Language (WLASL) and Microsoft American Sign Language (MS-ASL) datasets. The results show consistent performance for reduced vocabularies and across datasets, but accuracy decreases as the number of classes increases. The landmark representation also abstracts away identifying visual information, offering a privacy-conscious alternative to raw video, although the prototype remains a research system rather than a production-ready translator.

Keywords: Sign Language Recognition, Artificial Intelligence, Deep Neural Networks, Computer Vision, Hand Landmarks

Resumen

El reconocimiento de lengua de signos aborda barreras de comunicación entre personas sordas o con discapacidad auditiva y usuarios que no conocen la lengua de signos. Este Proyecto Fin de Grado desarrolla y evalúa un prototipo de investigación para el reconocimiento de palabras en American Sign Language (ASL) a partir de vídeos breves. El trabajo revisa primero el problema de accesibilidad, los métodos de visión por computador utilizados para el reconocimiento y los requisitos técnicos del sistema. A continuación, recoge la evolución metodológica del proyecto, desde los enfoques iniciales basados en detección de manos y reconocimiento estático de letras hacia un modelo basado en secuencias que representa el movimiento de los signos a lo largo de los fotogramas. El sistema final utiliza MediaPipe Hands para extraer 21 puntos clave 3D de la mano por fotograma, normaliza las secuencias resultantes y clasifica cada vídeo mediante una red recurrente con puertas (GRU) bidireccional con un mecanismo de atención temporal. Los experimentos evalúan las decisiones de arquitectura, la longitud de secuencia, la evaluación entre conjuntos de datos y el tamaño del vocabulario utilizando los conjuntos Word-Level American Sign Language (WLASL) y Microsoft American Sign Language (MS-ASL). Los resultados muestran un comportamiento consistente en vocabularios reducidos y entre conjuntos de datos, pero la precisión disminuye al aumentar el número de clases. La representación mediante puntos clave también descarta la mayor parte de la información visual del vídeo original, lo que limita los datos personales que conserva el sistema, aunque el prototipo sigue siendo un sistema de investigación y no un traductor listo para producción.

Palabras clave: reconocimiento de lengua de signos, inteligencia artificial, redes neuronales profundas, visión por computador, puntos clave de la mano

Contents

List of Acronyms **xiii**

1 Introduction **1**
1.1 Problem description . . . 1
1.2 Research scope and approach . . . 1
1.3 Outline . . . 2

2 State of the Art **3**
2.1 From Symbolic to Subsymbolic AI . . . 3
2.2 Deep Neural Networks . . . 4
2.3 Feature Extraction Architectures . . . 5
2.4 Sequence Processing Architectures . . . 8
2.5 Attention Mechanisms . . . 10
2.6 Related Work in Sign Language Recognition . . . 11
2.7 Datasets and Limitations . . . 12

3 Objectives **13**
3.1 Main objective . . . 13
3.2 Specific objectives . . . 13
3.3 Validation methods . . . 14

4 Methodology and Work Plan Overview **17**
4.1 Materials and Resources . . . 17
4.2 Methods . . . 18
4.3 Methodology . . . 20
4.4 Work Plan . . . 21
4.5 Constraints and Limitations . . . 24

5 Technical Development **25**
5.1 Solution design . . . 25
5.2 Requirements . . . 30
5.3 Datasets . . . 33

6 Experiments **39**
6.1 Experimental Methodology . . . 39
Part I: Quantitative Evaluation . . . 41

6.2 Phase 1: Exploratory Work and Methodological Pivot . . . 41
6.3 Architecture Selection on WLASL30 . . . 41
6.4 Sequence Length Sweep . . . 44
6.5 WLASL30 Validation . . . 45
6.6 Cross-Dataset Validation: MS-ASL50 . . . 46
6.7 Vocabulary Scaling . . . 48
6.8 Limitations . . . 49
Part II: Qualitative Evaluation . . . 51
6.9 Qualitative Validation . . . 51

7 Ethical Implications and Social Impact 53
7.1 Ethical Impact of AI-Based Systems . . . 53
7.2 Ethical Evaluation Framework: Google's AI Principles . . . 53
7.3 Project Analysis Against Google's AI Principles . . . 54
7.4 Impact Scale of Ethical Aspects . . . 57
7.5 Conclusions of the Ethical Analysis . . . 58

8 My Journey at UFV 60

9 Conclusiones 61
9.1 General Conclusions . . . 61
9.2 Conclusions regarding Project Objectives . . . 61
9.3 Next Steps . . . 61

Bibliography 62

A Additional Experimental Figures 65
A.1 Large Confusion Matrix . . . 65
A.2 Vocabulary Scaling Diagnostic . . . 67

List of Figures

2.1 Basic DNN structure. Inputs are transformed through hidden layers into an output prediction. Source: self-created. 4
2.2 Hierarchical feature representations learned by a CNN. Layer 1 responds to oriented edges and simple colour gradients; Layer 2 captures corners, circular patterns and basic textures; Layer 3 represents more complex textures and object parts. Adapted from Zeiler and Fergus [7]. 6
2.3 AlexNet architecture, showing stacked convolutional and pooling layers followed by fully connected layers. Source: Krizhevsky et al. [5]. 6
2.4 MediaPipe Hands pipeline. The palm detector locates the hand region on the first frame or when tracking is lost; the landmark model then regresses 21 three-dimensional landmarks inside the cropped region. Subsequent frames reuse the landmarks from the previous frame, avoiding running palm detection on every frame. Source: self-created, adapted from Zhang et al. [8]. 7
2.5 Unrolled recurrent layer. The same recurrent computation is applied at each time step while the hidden state carries information forward. Source: self-created. 8
2.6 Comparison of gated recurrent units. (a) LSTM cell with input, forget and output gates. (b) GRU cell with update and reset gates. Source: Chung et al. [11]. 9
2.7 Attention mechanism in an encoder-decoder architecture. The context vector is a weighted sum of encoder annotations. Source: Bahdanau et al. [13]. 11

4.1 Project Gantt chart showing the executed work plan grouped into seven phases, from project framing and initial hand-detection experiments to landmark-based word recognition, cross-dataset validation, prototype integration and final documentation. Source: self-created with GanttProject. 22

5.1 Solution architecture showing the data preparation, training and evaluation, and runtime inference stages. Source: self-created from the implemented pipeline. 26

6.1 Row-normalised confusion matrix of the selected GRU+Attention configuration on the WLASL30 test split (34 samples). Source: self-created from experiment outputs. 43
6.2 Per-metric distribution across the sequence-length sweep (3 seeds at 32, 64, and 96 frames). Source: self-created from experiment outputs. 45
6.3 Top-1 accuracy as a function of vocabulary size on WLASL. Source: self-created from experiment outputs. 49

A.1 Row-normalised confusion matrix on the MS-ASL50 test split for seed 2026 (236 samples). Source: self-created from experiment outputs. 66
A.2 Parallel-coordinates view of the selected GRU+Attention scaling line on WLASL. Each line is one experiment, coloured from green for smaller vocabularies to red for larger vocabularies. Source: self-created from experiment outputs. 68

List of Tables

3.1 Validation mapping for the specific objectives. 16

5.1 Operational environment for training, landmark extraction, and runtime inference. . . 29
5.2 FR-001: Landmark extraction from video clips 31
5.3 FR-002: Word-level sign classification 31
5.4 FR-003: Reproducible experiment tracking 31
5.5 FR-004: Multi-seed training support 32
5.6 NFR-001: Inference latency 32
5.7 NFR-002: Modular code architecture 32
5.8 NFR-003: Documentation completeness 33
5.9 Comparative overview of all datasets used in this FDP. 34

6.1 Single-split test results of the five locally tuned architectures evaluated on WLASL30 (34 test samples). Source: self-created from experiment outputs. Standard deviations are not available at this stage; the multi-seed validation of the selected configuration is in Section 6.5. 42
6.2 Sequence-length sweep on WLASL30 with 3 seeds per row. Mean ± standard deviation across seeds. Top-1 standard deviations are bounded below by the test-set granularity (≈2.94 pp per misclassification on 34 samples). Source: self-created from experiment outputs. 44
6.3 Per-seed test results of the selected GRU+Attention configuration on the fixed WLASL30 split (34 test samples). Source: self-created from experiment outputs. . . 45
6.4 Side-by-side comparison of the GRU+Attention pipeline on WLASL30 and MS-ASL50. Both rows are aggregated over 3 seeds. Source: self-created from experiment outputs. 47
6.5 Per-seed test results on MS-ASL50 (236 test samples). Source: self-created from experiment outputs. 47
6.6 Vocabulary scaling of the selected GRU+Attention configuration on WLASL subsets. The 30-class row is the multi-seed mean from Table 6.4; all other rows are single-seed runs and standard deviations cannot be reported. *Train time* is the total wall-clock time of the training run on the NVIDIA RTX 3050 Ti Laptop GPU described in Section 6.1. Source: self-created from experiment manifests. 48
6.7 Limitation categories, ordered from data acquisition constraints to modelling choices. Source: self-created from the experimental audit. 50
6.8 Usability questionnaire results for the SignToText demo. Means are computed on a 1–5 Likert scale. Source: self-created from Google Form responses. 52

6.9 Most valued SignToText features according to the usability questionnaire. Participants could select multiple options. Source: self-created from Google Form responses. 52

7.1 Ethical evaluation metrics based on Google's AI Principles [33] 58

List of Acronyms

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASL	American Sign Language
AWS	Amazon Web Services
BBox	Bounding Box
BiGRU	Bidirectional Gated Recurrent Unit
BPTT	Backpropagation Through Time
CEC	Constant Error Carousel
CNN	Convolutional Neural Network
CRISP-DM	Cross-Industry Standard Process For Data Mining
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
CV	Cross-Validation
FDP	Final Degree Project
FR	Functional Requirement
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
HTTP	Hypertext Transfer Protocol
IoU	Intersection Over Union
JSON	JavaScript Object Notation
JSONL	JavaScript Object Notation Lines
LTS	Long-Term Support
LSTM	Long Short-Term Memory
mAP	Mean Average Precision
ML	Machine Learning
MNIST	Modified National Institute Of Standards And Technology

Acronym	Meaning
MS-ASL	Microsoft American Sign Language
NFR	Non-Functional Requirement
OpenCV	Open Source Computer Vision Library
REST	Representational State Transfer
RGB	Red Green Blue
RNN	Recurrent Neural Network
RQ	Research Question
SLR	Sign Language Recognition
SO	Specific Objective
SSDLite	Single Shot Detector Lite
TTA	Test-Time Augmentation
UFV	Universidad Francisco De Vitoria
URL	Uniform Resource Locator
VRAM	Video Random-Access Memory
WLASL	Word-Level American Sign Language
XML	Extensible Markup Language
YAML	Human-Readable Data Serialisation Language
YOLO	You Only Look Once

Chapter 1

Introduction

Sign Language Recognition (SLR) brings together accessibility, computer vision and sequence modelling. Far from being mere gestural shorthand or direct encodings of spoken languages, sign languages are sophisticated natural languages with a unique visual-spatial structure. Meaning is conveyed through a complex interplay of handshape, movement, orientation, location, timing and, in many cases, non-manual cues such as facial expression or body posture. Automatic recognition from video is therefore a demanding problem: the same sign may vary across signers, camera viewpoints, execution speeds, lighting conditions and recording devices.

1.1 Problem description

This project studies American Sign Language (ASL) recognition as an accessibility-oriented research problem. Unlike static image classification, sign recognition is inherently temporal: a single frame may show the handshape and approximate location of a sign, but it cannot represent the direction of movement, the order of articulatory changes or the rhythm of execution. The sign must therefore be interpreted from the ordered relationship between frames.

The task addressed in this work is isolated word recognition. The system receives a short video clip and must assign it to a word gloss, that is, a written label used to annotate a sign. This task is more limited than continuous sign language translation, but it preserves the temporal nature of signs and provides a suitable scope for a research-oriented undergraduate project.

1.2 Research scope and approach

The central research question of this work is whether compact visual representations can provide enough information for temporal models to classify short ASL video clips into word glosses. The project evaluates different neural architectures under reproducible experimental conditions, with the objective of identifying a configuration that balances accuracy and computational cost. The work does not aim to replace professional interpreters or to deliver a production-ready translator.

The experimental work is grounded on public ASL datasets, mainly WLASL [1] and MS-ASL [2]. A mobile and server-based runtime prototype is also developed as a proof of concept, although the

main scientific contribution is the experimental evaluation of the recognition approach rather than the deployment of an assistive product.

1.3 Outline

The remainder of the work is organised as follows:

- Chapter 2 reviews the State of the Art, providing the theoretical foundation in computer vision and deep learning required for this work. It examines convolutional neural networks and pose estimation, specifically MediaPipe Hands, before addressing temporal modelling through recurrent networks, Long Short-Term Memory (LSTM) layers, Gated Recurrent Unit (GRU) layers, Transformers and attention mechanisms. The chapter also evaluates previous research in SLR to establish the context for the current approach.
- Chapter 3 defines the main objective, the specific objectives, and the validation methods used to determine whether each objective has been achieved.
- Chapter 4 outlines the methodology and project work plan. It details the iterative, experiment-driven approach adopted throughout development and delineates the main project phases. Finally, it addresses the technical resources and constraints that influenced the scope and design of the project.
- Chapter 5 presents the technical development and system architecture. It provides a rigorous account of the data preprocessing pipeline and the experimental framework established for training and evaluation. The chapter concludes with a description of a functional proof of concept, demonstrating a real-time implementation through a mobile client and a FastAPI-based inference server [3].
- Chapter 6 reports the experimental evaluation. It includes an architecture comparison and sequence-length analysis, followed by validation on the WLASL30 and MS-ASL50 datasets. The chapter also addresses cross-dataset validation, vocabulary scaling and the qualitative validation design, and concludes with a critical discussion of the main limitations observed during experimentation.
- Chapter 7 analyses the ethical implications and social impact of the project, structured around the Google Artificial Intelligence Principles and ethical framework. It provides a critical examination of accessibility, privacy and algorithmic bias, while addressing the requirements for responsible deployment and the importance of reproducibility. Finally, it evaluates the broader social value of assistive technologies through these ethical guidelines.
- Chapter 8 reflects on the academic journey at Universidad Francisco de Vitoria and relates the project to the competences developed during the degree.
- Chapter 9 presents the conclusions and future work, summarising the main findings, the limitations of the current prototype, and possible directions for extending the research.

Chapter 2

State of the Art

This chapter reviews the technical foundations that support this project. It begins with the move from symbolic Artificial Intelligence (AI) to data-driven learning, then introduces Deep Neural Networks (DNNs), feature extraction architectures, sequence processing layers, attention mechanisms, related Sign Language Recognition (SLR) work and the datasets most relevant to American Sign Language (ASL) recognition.

2.1 From Symbolic to Subsymbolic AI

Early AI systems were symbolic, representing knowledge through explicit logic and predefined rule sets. They solved problems by applying human-authored rules, often organised as conditional decision trees. While this paradigm flourished within constrained environments, it proved inadequate for perception tasks, such as computer vision and speech, due to the inherent noise, ambiguity, and high-dimensionality of real-world data. This phenomenon, termed the "knowledge acquisition bottleneck", implies that as task complexity scales, the maintenance of handcrafted rule bases becomes untenable.

In the context of sign language, symbolic approaches could encode basic gesture prototypes, but they failed to capture the nuanced articulatory variation found across signers, camera angles and execution speeds. A fixed rule for a hand shape may describe an idealised sign, yet the observed motion in video changes with signer anatomy, partial occlusion, lighting and the temporal rhythm of the execution. These sources of variation make exhaustive symbolic description impractical.

Subsymbolic AI addresses the same class of problems through statistical learning. Instead of hard-coding expert rules for every possible case, a learning system estimates parameters from examples. This transition was driven by advances in neural networks and statistical learning theory, culminating in deep learning methods that learn hierarchical features directly from raw inputs [4]. By treating perception as a high-dimensional optimisation problem, these systems reduce the noise and ambiguity that previously led to the knowledge acquisition bottleneck.

In the SLR domain, this transition mirrors the evolution of the field: moving away from attempts to define a perfect hand-shape rules and towards architectures that extract latent features from raw video data. Consequently, the burden of feature engineering shifts from the human expert to the learning

algorithm, allowing for a more fluid and robust interpretation of manual dynamics that aligns with the success seen in modern speech recognition and object detection frameworks.

2.2 Deep Neural Networks

A neural network is a parameterised function composed of layers. Each layer receives an input vector, applies a linear transformation and passes the result through a non-linear activation function. A DNN stacks several such layers, allowing the model to compose simple transformations into progressively more abstract representations. Figure 2.1 shows this structure as a mapping from input features to an output prediction through hidden layers.

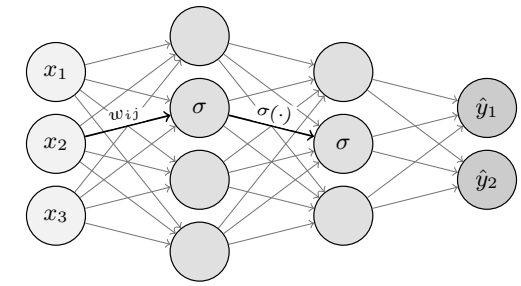


Figure 2.1: Basic DNN structure. Inputs are transformed through hidden layers into an output prediction. Source: self-created.

For a layer l, the computation can be written as:

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) \tag{2.1}$$

where $\mathbf{W}^{(l)}$ contains the learned weights, $\mathbf{b}^{(l)}$ is the bias vector and σ is the activation function. The Rectified Linear Unit (ReLU), defined as $\text{ReLU}(x) = \max(0, x)$, is a common activation because it introduces non-linearity while keeping the computation simple. The output layer depends on the task: classification layers estimate class probabilities, whereas regression layers estimate continuous quantities such as coordinates.

Training adjusts the parameters $\theta = \{\mathbf{W}, \mathbf{b}\}$ so that the function f_θ maps inputs to the desired targets. The model computes a prediction, compares it with the ground truth through a loss function and updates the parameters by gradient-based optimisation. In this sense, a DNN is not a list of explicit rules but an optimised function approximator.

This formulation explains the link between DNNs and feature extraction. Intermediate layers do not only transform the data numerically; they also produce representations that make the target task easier. In computer vision, these representations may correspond to edges, textures, object parts or pose coordinates. Feature extraction therefore denotes the process of converting raw observations into informative variables that a later classifier or sequence model can use.

2.3 Feature Extraction Architectures

Feature extraction architectures convert video frames into representations that preserve relevant visual information while reducing irrelevant variation. In SLR, two families are especially important. Pixel-based techniques operate directly on image tensors, where each pixel stores colour or intensity values. Pose estimation techniques first detect anatomical points and then represent the signer through coordinates. The distinction matters because it determines whether the model learns from appearance information or from a geometric abstraction of the body and hands.

2.3.1 Pixel-Based Techniques

A digital image can be represented as a tensor $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, where H and W are height and width and C is the number of colour channels. Pixel-based techniques use this tensor as the direct input to the model. This preserves all visual information, including hand texture, background, illumination and camera artefacts.

Convolutional Neural Networks (CNNs) are the standard DNN architecture for pixel-based computer vision [4]. A convolutional layer applies a small matrix of learned parameters, called a kernel or filter, across local regions of the image. At each spatial position, the kernel computes a response that indicates whether a visual pattern is present. The collection of responses produced by one kernel is a feature map.

For input tensor $\mathbf{x}$ and output feature map $\mathbf{y}$, a convolutional layer can be written as:

$$\mathbf{y}_{i,j,k} = \sigma\left(\sum_{c=1}^{C}\sum_{u,v}\mathbf{w}_{u,v,c,k}\mathbf{x}_{i+u,j+v,c} + b_k\right) \tag{2.2}$$

where $\mathbf{w}$ is the set of kernels, b_k is the bias for feature map k and σ is the activation function. Because the same kernel is reused across the image, convolutional layers encode translation equivariance: if a pattern shifts in the image, its corresponding activation shifts in the feature map.

CNN architectures stack convolutional, activation and pooling layers. Pooling reduces the spatial resolution of feature maps by summarising local neighbourhoods, commonly through maximum or average values. This supports local translation invariance, since small shifts in the input produce similar pooled activations. Early layers usually respond to oriented edges and simple textures; deeper layers combine those responses into larger shapes and object-level cues, as illustrated in Figure 2.2. AlexNet [5] and ResNet [6] are established examples of this hierarchical design.

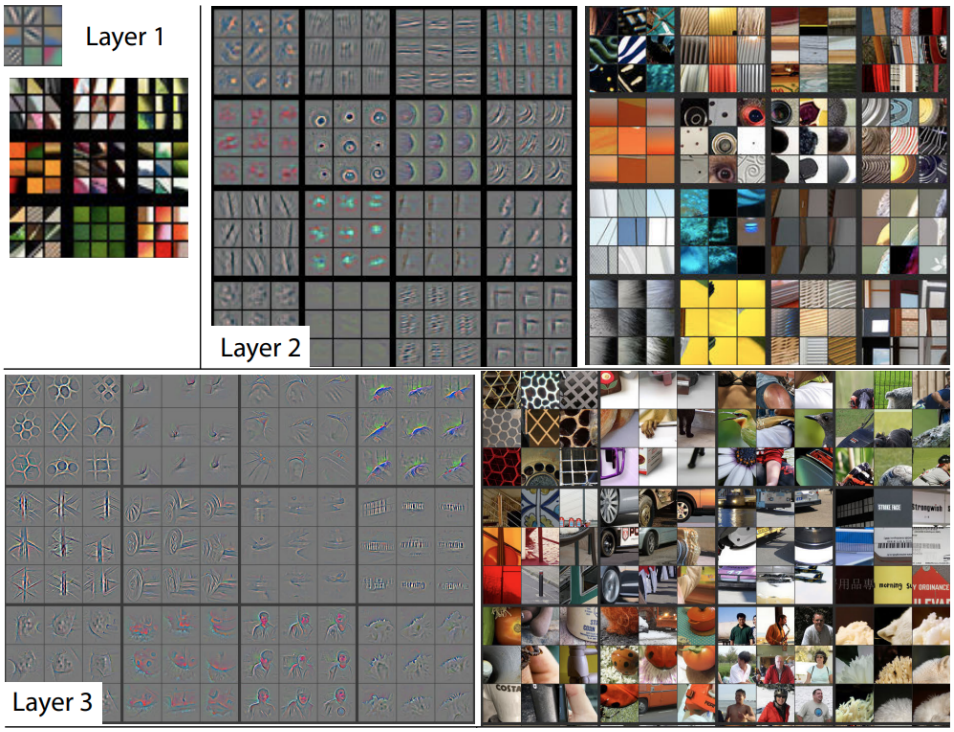


Figure 2.2: Hierarchical feature representations learned by a CNN. Layer 1 responds to oriented edges and simple colour gradients; Layer 2 captures corners, circular patterns and basic textures; Layer 3 represents more complex textures and object parts. Adapted from Zeiler and Fergus [7].

Figure 2.3 shows the AlexNet architecture. The diagram is relevant here because it illustrates the repeated pattern of convolution, pooling and fully connected layers that made deep pixel-based recognition practical.

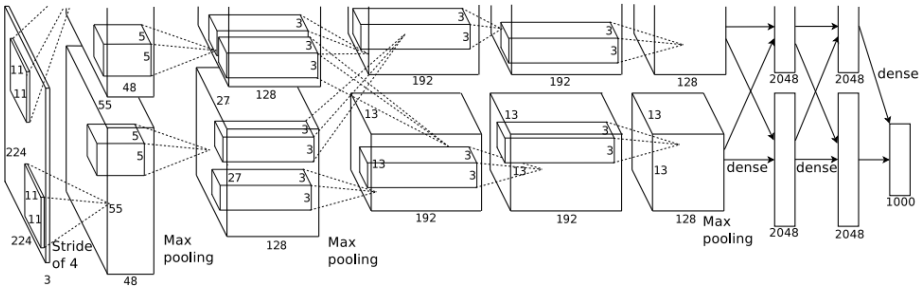


Figure 2.3: AlexNet architecture, showing stacked convolutional and pooling layers followed by fully connected layers. Source: Krizhevsky et al. [5].

In SLR, pixel-based CNNs can learn discriminative visual patterns directly from frames. Their main

limitation is that they also receive information that may not define the sign, including background, skin tone and recording conditions. When the available dataset is limited, this can encourage the model to fit appearance correlations instead of sign structure.

2.3.2 Pose Estimation Techniques

Pose estimation is the task of estimating the spatial location of anatomical points from an image or video frame. In machine learning, regression refers to predicting continuous numerical values rather than selecting one class from a fixed set. A landmark is a tracked point with image or world coordinates. In hand pose estimation, many landmarks correspond to joints, which are anatomical articulations such as knuckles, while fingertip landmarks mark the end of each finger. A regression-based pose estimator therefore predicts landmark coordinates rather than a discrete class label.

Pose pipelines are commonly organised as top-down or bottom-up systems. A top-down pipeline first detects the relevant object or body part, then estimates landmarks inside the detected region. A bottom-up pipeline first predicts candidate landmarks across the image, then groups them into instances. Both approaches convert raw image evidence into a structured geometric representation.

MediaPipe Hands [8] is a widely used hand pose estimation system. It uses the top-down structure shown in Figure 2.4: a hand detector proposes a region of interest, and a landmark model estimates hand coordinates inside that cropped region.

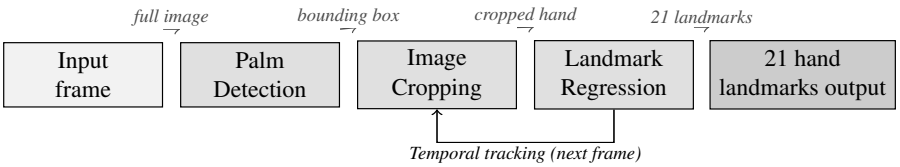


Figure 2.4: MediaPipe Hands pipeline. The palm detector locates the hand region on the first frame or when tracking is lost; the landmark model then regresses 21 three-dimensional landmarks inside the cropped region. Subsequent frames reuse the landmarks from the previous frame, avoiding running palm detection on every frame. Source: self-created, adapted from Zhang et al. [8].

The process consists of the following stages:

1. **Palm detection. Input:** full image frame. **Operation:** a single-shot detector estimates a bounding region around the palm. **Output:** a rectangle defining the hand region of interest.
2. **Image cropping. Input:** original frame and detected rectangle. **Operation:** the image is cropped and aligned around the hand. **Output:** a normalised hand image for the landmark model.
3. **Landmark regression. Input:** cropped hand image. **Operation:** a DNN regresses the coordinates of 21 hand landmarks. **Output:** a structured set of three-dimensional landmark coordinates.
4. **Temporal tracking. Input:** landmarks and region of interest from the previous frame. **Operation:** the pipeline reuses recent location information when the hand remains visible. **Output:** a frame-to-frame landmark sequence.

For SLR, landmark sequences provide a compact description of hand shape and movement. They remove much of the raw appearance signal and make it possible to derive additional geometric features such as bone vectors, joint angles and distances between fingers. The cost of this abstraction is that pose estimators may fail under occlusion, motion blur or unusual camera angles, and hand-only landmarks omit non-manual cues such as facial expression and upper-body movement.

2.4 Sequence Processing Architectures

Isolated sign recognition is a temporal classification problem. The input is not a single image but an ordered sequence of feature vectors, one per frame or sampled time step. A sequence processing layer therefore receives $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T\}$ and produces a representation that accounts for temporal order.

2.4.1 Recurrent Layers

Recurrent Neural Networks (RNNs) process a sequence by maintaining a hidden state. At time t, the layer combines the current input $\mathbf{x}_t$ with the previous hidden state $\mathbf{h}_{t-1}$:

$$\mathbf{h}_t = \phi(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \tag{2.3}$$

where ϕ is an activation function. The same parameters are shared at every time step, as shown in Figure 2.5. This sharing lets the layer process sequences of different lengths while applying the same transition rule throughout the video.

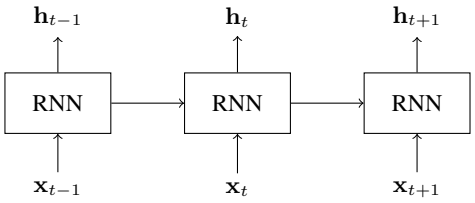


Figure 2.5: Unrolled recurrent layer. The same recurrent computation is applied at each time step while the hidden state carries information forward. Source: self-created.

Training recurrent layers uses Backpropagation Through Time (BPTT), which unfolds the recurrent computation across the sequence and propagates the loss backwards through the unfolded graph. As the number of time steps increases, standard recurrent layers frequently encounter the vanishing gradient problem [9]. The gradient signal becomes too small to preserve information from early time steps, which limits long-range temporal modelling.

The recurrent layer above is the simple recurrent case. The following advanced layer types are described in the next subsections:

- Long Short-Term Memory (LSTM) layers, which introduce a cell state and gates to regulate information flow.
- Gated Recurrent Unit (GRU) layers, which simplify the LSTM design while retaining gated memory.

- Transformer layers, which process all positions through self-attention rather than recurrence.

2.4.2 Long Short-Term Memory Layers

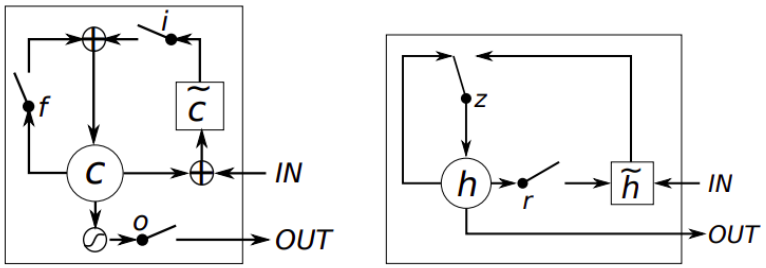
Long Short-Term Memory (LSTM) layers were proposed by Hochreiter and Schmidhuber [10] to improve the modelling of long temporal dependencies. An LSTM layer maintains two vectors: the hidden state $\mathbf{h}_t$, which is exposed to the next layer or output, and the cell state $\mathbf{C}_t$, which carries memory along the sequence.

The LSTM cell regulates this memory through gates. The forget gate controls which information is removed from the cell state, the input gate controls which new information is written and the output gate controls which part of the cell state becomes the hidden state. Figure 2.6 (a) represents these gates as f, i and o, and shows how the cell state C provides a direct path for information flow.

This gated memory path is the mechanism referred to as the Constant Error Carousel. It reduces the decay of the error signal during BPTT and allows the layer to retain information over longer time spans than a simple recurrent layer. The cost is computational: each time step requires several linear transformations, increasing both parameter count and training time.

2.4.3 Gated Recurrent Unit Layers

Gated Recurrent Unit (GRU) layers were introduced by Chung et al. [11] as a simpler gated recurrent alternative. A GRU layer removes the separate cell state and stores memory directly in the hidden state $\mathbf{h}_t$. It uses two gates, shown in Figure 2.6(b): the update gate z and the reset gate r.



(a) Long Short-Term Memory (b) Gated Recurrent Unit

Figure 2.6: Comparison of gated recurrent units. (a) LSTM cell with input, forget and output gates. (b) GRU cell with update and reset gates. Source: Chung et al. [11].

The GRU gate equations at time t are:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z) \tag{2.4}$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r) \tag{2.5}$$

The reset gate controls how much previous information is used to compute the candidate hidden state $\tilde{\mathbf{h}}_t$. The update gate controls the interpolation between the previous hidden state and the candidate state:

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \tag{2.6}$$

When $\mathbf{z}_t$ is close to zero, the layer copies most of the previous hidden state. When it is close to one, the layer replaces it with new information. This gives GRU layers a compact mechanism for modelling temporal dependencies with fewer parameters than LSTM layers.

2.4.4 Transformer Layers

Transformer layers replace recurrence with self-attention [12], an extension of the attention mechanism described in Section 2.5. Instead of processing the sequence step by step, a Transformer computes interactions between all positions in parallel. Each input position is projected into a query, a key and a value vector. The similarity between queries and keys determines how much each position attends to the others, and the values are combined according to those attention weights.

This design supports parallel computation across the sequence, which is useful for large datasets and long inputs. However, Transformers do not include the same sequential inductive bias as recurrent layers. They require positional information to represent order, and their performance in small SLR datasets may depend strongly on the amount of data available for training. For this reason, Transformer layers are relevant to the comparison of sequence models, but they do not replace recurrent layers by default.

2.5 Attention Mechanisms

Attention mechanisms assign different weights to different parts of an input sequence. In neural machine translation, Bahdanau et al. [13] introduced attention to avoid compressing a full source sentence into a single fixed-length vector. The same idea applies to video recognition: not every frame contributes equally to the final prediction.

Given encoder states $\{\mathbf{h}_t\}_{t=1}^{T}$, an attention module computes a score e_t for each time step and normalises the scores with a softmax function:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^{T} \exp(e_k)}, \quad e_t = a(\mathbf{s}, \mathbf{h}_t) \tag{2.7}$$

where $a(\cdot)$ is a scoring function and α_t is the attention weight assigned to time step t. The context vector is then computed as:

$$\mathbf{c} = \sum_{t=1}^{T} \alpha_t \mathbf{h}_t \tag{2.8}$$

Figure 2.7 shows this weighted combination: the decoder state attends to encoder annotations through weights $\alpha_{t,1}, \alpha_{t,2}, \ldots, \alpha_{t,T}$.

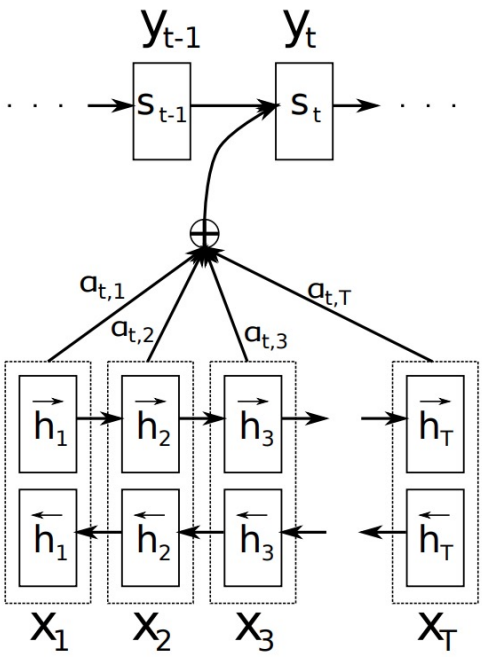


Figure 2.7: Attention mechanism in an encoder-decoder architecture. The context vector is a weighted sum of encoder annotations. Source: Bahdanau et al. [13].

For example, in an SLR clip of the sign *drink*, the first frames may show the hand moving towards the mouth, the central frames may contain the class-defining motion and the final frames may show the hand returning to rest. Temporal attention can assign higher weights to the central frames and lower weights to the preparation and recovery frames. This does not remove the need for a sequence model; it adds a learned weighting stage over the temporal representation produced by that model.

2.6 Related Work in Sign Language Recognition

SLR research combines object detection, feature extraction and temporal classification. Earlier vision pipelines often treated hands as regions to be detected before classification. Modern object detectors such as Faster Region-based Convolutional Neural Network (Faster R-CNN) [14], Single Shot MultiBox Detector (SSD) [15], EfficientDet [16] and the You Only Look Once (YOLO) family [17], [18] frame this problem as locating objects and assigning class scores in images. YOLO is especially relevant for real-time hand detection because it performs detection in a single forward pass, and YOLOv8 is a current implementation frequently used in practical detection pipelines [19].

Survey work distinguishes between isolated sign recognition, continuous sign recognition and sign language translation [20], [21], [22]. Isolated recognition assigns a single gloss to a trimmed clip, which is the scope of this project. Continuous recognition must segment and classify signs from longer streams, while translation must produce spoken-language text and account for grammar, context and non-manual markers.

Word-level ASL recognition is strongly associated with WLASL and MS-ASL. Li et al. [1] introduced WLASL together with benchmark comparisons for word-level recognition from video, while Joze and Koller [2] introduced MS-ASL as a larger ASL benchmark. Both works helped move isolated SLR evaluation away from small curated vocabularies and towards larger, noisier video collections.

Architecturally, the literature has moved from handcrafted descriptors towards DNN-based pipelines. Pixel-based CNNs learn frame-level visual features, recurrent layers model temporal evolution and attention modules weight the most informative time steps. Transformer-based work such as Sign Language Transformers [23] and STMC-Transformer [24] shows how attention-based sequence models have been applied to continuous recognition and translation settings.

Pose-based methods replace raw frames with landmark sequences, reducing dimensionality and limiting the influence of background appearance. Bohacek and Hruz [25] proposed a pose-based Transformer for word-level SLR, while Sign2Pose [26] combines pose features and Transformer-based gloss prediction on WLASL. Large-scale work such as BSL-1K [27] also highlights that dataset scale, weak supervision and cross-dataset transfer remain central concerns in sign recognition research. This project is situated in the pose-based line: MediaPipe Hands provides hand landmarks, and the recognition model evaluates recurrent and attention-based temporal processing.

2.7 Datasets and Limitations

SLR systems depend strongly on the datasets available for training and evaluation. Unlike general image recognition, sign language recognition still lacks abundant large-scale benchmarks across languages, domains and signer populations. Dataset design therefore shapes what a recognition model can learn: the vocabulary determines the classification space, recording conditions affect visual variation and annotation practice defines the gloss labels used for evaluation.

WLASL [1] and MS-ASL [2] are the main ASL references for word-level video recognition. They provide public benchmarks at a scale suitable for comparing temporal models, but they also reflect common limitations in SLR research. Many clips originate from online video sources, which introduces variable camera quality, inconsistent clip duration and dataset attrition when source videos are removed or made private. Class imbalance also remains a recurring issue, since frequent signs have many more examples than rare signs.

Auxiliary resources such as static alphabet datasets or hand-detection corpora are useful for engineering stages such as detector training or letter-level experiments. They should not be treated as word-level SLR benchmarks because they do not evaluate temporal sign dynamics. For that reason, this chapter treats them as secondary resources and leaves the dataset composition and preparation details to the technical development chapter.

Chapter 3

Objectives

This chapter defines the purpose of the project, decomposes it into specific and verifiable objectives and states how each objective is validated in the remainder of the document. The objectives describe the research and project outcomes pursued by the work. They are therefore different from the software requirements presented in Section 5.2, which specify the functional and non-functional behaviour expected from the implemented system.

3.1 Main objective

The main objective of this Final Degree Project (FDP) is to design, implement and experimentally validate a research prototype for word-level American Sign Language (ASL) recognition from short video clips through computer vision and neural modelling techniques. The purpose is not to build a production-ready translator, but to study how convolutional and temporal recognition approaches can support feasible and reproducible isolated-word sign recognition.

The hypothesis derived from this objective is that a compact sequence representation of sign movement can retain enough discriminative information to recognise a controlled vocabulary of isolated ASL signs while reducing dependence on raw video appearance. This hypothesis is tested by comparing candidate architectures, selecting the best-performing configuration and evaluating its behaviour across repeated runs, datasets and vocabulary sizes.

3.2 Specific objectives

The main objective is decomposed into six specific objectives. Each one is defined as a measurable project outcome and is later mapped to a validation method in Section 3.3.

- **SO1: Analyse state-of-the-art techniques in computer vision and temporal modelling.** This objective covers the relevant foundations of computer vision, pose estimation, MediaPipe Hands, sequence modelling, convolutional architectures, recurrent layers, attention mechanisms and Transformer architectures, with emphasis on their applicability to word-level sign language recognition.

- **SO2: Define a landmark-based data representation for ASL video clips.** This objective focuses on transforming raw ASL video clips into normalised sequences of three-dimensional hand landmarks suitable for temporal modelling, while documenting the datasets, preprocessing decisions and limitations introduced by landmark extraction. The resulting representation is intended to provide a compact and consistent description of hand movement and spatial relationships across frames, facilitating the development of sign language recognition models while reducing sensitivity to background variation, lighting conditions and camera position.
- **SO3: Design and compare candidate convolutional and temporal architectures for word-level ASL recognition.** This objective evaluates alternative architectures under a controlled 30-class subset of the Word-Level American Sign Language (WLASL) dataset, referred to as WLASL30, and identifies the configuration most suitable for the available data regime. It also examines each architecture's ability to capture temporal dependencies and dynamic hand movements from landmark sequences, while assessing performance, generalisation, computational cost and consistency across different sign categories. The objective further analyses the trade-off between model complexity and recognition accuracy in order to determine the most appropriate architecture for reliable word-level ASL recognition.
- **SO4: Evaluate the selected recognition pipeline through reproducible experimental protocols.** This objective evaluates the selected model using standard classification metrics and protocols that reduce dependence on a single random seed or split, including multi-seed evaluation and stratified cross-validation. It also assesses the stability, consistency and generalisation capability of the recognition pipeline across different evaluation settings to ensure that the reported performance is reliable and reproducible.
- **SO5: Analyse the generalisation and scalability of the recognition pipeline.** This objective tests whether the selected approach transfers effectively to the 50-class subset of the independent Microsoft American Sign Language (MS-ASL) dataset, referred to as MS-ASL50, and analyses how performance changes as the WLASL vocabulary size increases. It also evaluates the stability and scalability of the recognition pipeline under more challenging classification settings, identifying potential limitations related to vocabulary expansion, dataset variability and model generalisation across different signing conditions.
- **SO6: Evaluate the runtime proof of concept and the ethical implications of the recognition approach.** This objective integrates the recognition pipeline into an end-to-end mobile/server prototype as a proof of concept and analyses its accessibility motivation, privacy implications, bias risks, reproducibility constraints and responsible-deployment requirements.

3.3 Validation methods

Table 3.1 maps each specific objective to the method used to validate it and to the document evidence where the committee can verify the result. Detailed numerical results are intentionally left to the Experiments chapter; this section only defines how objective achievement is assessed.

The experimental objectives use four classification metrics. **Top-1 accuracy** measures the proportion of test samples for which the highest-scoring prediction matches the ground-truth label. For example, if 20 out of 34 WLASL30 test clips are classified correctly as the first prediction, Top-1 accuracy is $20/34 = 58.82\%$. **Top-5 accuracy** measures whether the ground-truth label appears among the five highest-scoring predictions. For example, if the correct ASL sign appears in the five

proposed labels for 27 out of 34 clips, Top-5 accuracy is $27/34 = 79.41\%$.

Macro-F1 evaluates class-level behaviour by computing the F1 score for each class and then averaging all classes with equal weight. F1 combines precision and recall as $2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$; for example, a class with precision 0.80 and recall 0.60 obtains an F1 score of 0.69. Macro-F1 is therefore useful when frequent classes could otherwise dominate the average. **Balanced accuracy** averages recall across classes. For example, if three classes obtain recall values of 1.00, 0.50 and 0.00, the balanced accuracy is $(1.00 + 0.50 + 0.00)/3 = 0.50$.

The evaluation protocols complement these metrics. A **multi-seed evaluation** trains the same configuration with different random seeds and reports the mean and standard deviation, which helps separate model behaviour from random initialisation effects. **Stratified cross-validation** splits the dataset into folds while preserving class proportions, so that the evaluation does not depend on a single train-test partition.

Table 3.1 indicates, for each specific objective, which of the metrics defined above are used and where the corresponding evidence is presented in the document.

Table 3.1: Validation mapping for the specific objectives.

Objective	Validation method and metrics	Evidence in the document
SO1	The objective is validated by presenting a structured review of the main techniques needed to understand the proposed system, including convolutional architectures, pose estimation, MediaPipe Hands, recurrent layers, attention mechanisms and Transformer architectures.	State of the Art chapter (Chapter 2) and the cited literature used to justify the methodological choices.
SO2	The objective is validated by documenting the pipeline that converts ASL videos into normalised landmark sequences and by describing the datasets, extraction process, preprocessing steps and data-quality constraints.	System design and dataset description in Section 5.1 and Section 5.3; methodological pivot in Section 6.2.
SO3	The objective is validated by comparing candidate convolutional and temporal architectures on the same WLASL30 benchmark using Top-1 accuracy, Top-5 accuracy, Macro-F1 and balanced accuracy, followed by selection of the best-performing configuration under the controlled comparison.	Architecture selection in Section 6.3 and Table 6.1.
SO4	The objective is validated through the multi-seed protocol, the sequence-length sweep, stratified 5-Fold Cross-Validation and the reporting of Top-1 accuracy, Top-5 accuracy, Macro-F1 and balanced accuracy across repeated runs.	Experimental methodology in Section 6.1; sequence-length sweep in Table 6.2; WLASL30 validation in Section 6.5 and Table 6.3.
SO5	The objective is validated by retraining and evaluating the same pipeline on MS-ASL50 without architectural changes and by measuring Top-1 accuracy, Top-5 accuracy, Macro-F1 and balanced accuracy across progressively larger WLASL vocabularies.	Cross-dataset validation in Section 6.6, Table 6.4 and Table 6.5; vocabulary scaling in Section 6.7 and Table 6.6.
SO6	The objective is validated by describing the runtime prototype architecture, defining a qualitative validation protocol for live use and evaluating the ethical and social implications of the system as a research prototype.	Runtime architecture in Section 5.1; qualitative protocol in Section 6.9; ethical and social-impact analysis in Chapter 7.

Chapter 4

Methodology and Work Plan Overview

This chapter presents the methodology followed throughout the project, outlining the resources and tools employed, the technical approaches adopted at different stages, the workflow used to guide development and the sequence of activities undertaken. It also discusses the practical and technical constraints that influenced the decisions made during implementation. The technical development of the system is described in Section 5.1 and following, while the experimental evidence is reported in Section 6.1 and following.

4.1 Materials and Resources

The project required three resource categories: (1) Hardware; (2) Software Stack; and (3) Datasets. Each category is described below and cross-referenced to the chapter section where the technical detail is reported.

4.1.1 Hardware

All training, evaluation and landmark extraction were performed on a single local workstation whose specification is the operational environment of Section 5.1.5. The full table of components and version constraints is Table 5.1 and is not repeated here. The relevant hardware resource for the work plan is the GPU, an NVIDIA RTX 3050 Ti Laptop with 4 GB of VRAM. The practical implications of this memory ceiling are discussed in Section 4.5.

4.1.2 Software Stack

The training and evaluation pipeline runs on Python 3.10 with PyTorch 2.x and CUDA 12.x. Pose estimation uses MediaPipe Hands 0.10+, video decoding uses OpenCV 4.8+, and version control is provided by Git with GitHub as remote host. The mobile client is built with React Native and Expo. The inference server uses FastAPI 0.111 with Uvicorn 0.29. Documentation is written in LaTeX. The exact version constraints are locked in `pyproject.toml` for the training package and in

`04_server/requirements.txt` for the inference server, so that the operational environment can be reconstructed from a clean clone of the repository (cross-reference: Section 5.1.5 and the reproduction guide at `docs/reproducibility_guide.md`).

4.1.3 Datasets

The project used six datasets, grouped by their role in the development process.

1. **EgoHands.** EgoHands provides annotated hand images recorded from egocentric video. It was used during the initial hand-localisation stage to train and evaluate a detector capable of finding hands before any sign-recognition model was applied.
2. **Kaggle Hand Dataset.** The Kaggle Hand Dataset supplied additional hand images for the same localisation stage. It increased the visual variability of the detector training data and complemented EgoHands with examples from different capture conditions.
3. **Sign Language MNIST.** Sign Language MNIST contains static greyscale images of alphabet handshapes. It was used only in the exploratory letter-classification stage, where the objective was to test whether a simple image classifier could recognise isolated hand configurations.
4. **ASL Alphabet.** The ASL Alphabet dataset contains static colour images of American Sign Language (ASL) alphabet signs. It supported the same early letter-recognition branch as Sign Language MNIST, but it did not form part of the final word-level recognition pipeline because it does not contain temporal sign videos.
5. **WLASL.** The Word-Level American Sign Language (WLASL) dataset is the main video benchmark used for word-level ASL recognition. The project builds WLASL30 for controlled architecture comparison and also uses larger WLASL subsets to analyse vocabulary scaling.
6. **MS-ASL.** The Microsoft American Sign Language (MS-ASL) dataset is an independent word-level ASL video dataset reconstructed from public video sources. The project uses MS-ASL50 as a cross-dataset validation benchmark to test whether the selected recognition pipeline generalises beyond WLASL.

4.2 Methods

This section describes the methods used to carry out the work introduced in Chapter 1. It covers the theoretical basis, algorithms, metrics, exploratory case studies and replication support used in the project. Exact hardware specifications are reported in Section 5.1.5; exact experimental hyperparameters are reported in Section 6.1.

4.2.1 Theoretical and Algorithmic Basis

The final recognition method is based on supervised sequence classification. Each input sample is a short ASL video clip, and the output is a gloss label from a fixed vocabulary. The method combines pose estimation, recurrent sequence modelling and temporal attention.

Pose estimation provides the input representation. A landmark is a three-dimensional keypoint that represents a specific position on the detected hand, such as a fingertip or joint. MediaPipe Hands estimates 21 landmarks per detected hand, so each valid frame is represented as 63 scalar features.

This converts the recognition problem from pixel-based video classification into classification over structured skeletal sequences.

The sequence model uses Gated Recurrent Unit (GRU) layers for temporal modelling. A bidirectional GRU (BiGRU) layer processes the landmark sequence in forward and reverse temporal order, then combines both hidden-state streams. Training recurrent layers uses Backpropagation Through Time (BPTT), which unfolds the sequence across time steps so that the loss can update the recurrent weights. The selected architecture adds a temporal attention layer over the BiGRU outputs and uses the resulting context vector for classification. Throughout the document, this full architecture is referred to as the GRU+Attention model.

4.2.2 Recognition Pipeline

The implemented recognition pipeline has four stages:

1. **Video acquisition and decoding.** The input is a labelled video clip from WLASL, MS-ASL or the runtime prototype. OpenCV decodes the clip frame by frame. The output is an ordered frame sequence that preserves the temporal structure of the sign.
2. **Landmark extraction and normalisation.** MediaPipe Hands estimates 21 three-dimensional landmarks per frame. The coordinates are wrist-centred and length-normalised by the wrist-to-middle-finger distance. Frames where MediaPipe fails are linearly interpolated from neighbouring valid frames, and clips with a detection rate below 0.3 are discarded. The output is a per-clip `.npy` array of shape $(T, 21, 3)$ plus metadata.
3. **Training-time augmentation.** During training, the landmark coordinates are transformed through rotation, scaling, translation, additive Gaussian noise and horizontal mirroring. These operations simulate variation in signer position and execution while preserving the skeletal representation.
4. **Sequence classification.** The GRU+Attention architecture receives the landmark sequence and produces class scores over the selected gloss vocabulary. The same training loop also supports the alternative architectures compared in Section 6.3, so the comparison changes the sequence-modelling component while keeping the input representation fixed.

4.2.3 Training, Metrics and Validation

Training uses AdamW optimisation, cross-entropy loss with label smoothing, cosine learning-rate annealing and early stopping. The numerical values of these hyperparameters are reported in the experimental setup of Section 6.1.

The evaluation metrics are Top-1 accuracy, Top-5 accuracy, Macro-F1 and balanced accuracy. Top-1 accuracy measures whether the highest-scoring prediction matches the ground truth. Top-5 accuracy measures whether the ground truth appears within the five highest-scoring predictions. Macro-F1 averages per-class F1 scores without weighting by class frequency, while balanced accuracy averages per-class recall.

Validation uses Stratified 5-Fold Cross-Validation, multi-seed evaluation, sequence-length analysis and cross-dataset validation. Stratified 5-Fold Cross-Validation preserves the per-class distribution across folds. Multi-seed evaluation repeats the same configuration under different random seeds and reports mean and standard deviation. Sequence-length analysis tests the effect of the temporal input

window. Cross-dataset validation retrains the same pipeline on MS-ASL50 to assess generalisation beyond WLASL.

4.2.4 Exploratory Case Studies

Two exploratory case studies preceded the final landmark-based method. The first used You Only Look Once version 8 nano (YOLOv8n) for hand localisation on EgoHands and the Kaggle Hand Dataset. The second used Convolutional Neural Network (CNN) classifiers for static ASL letter recognition on Sign Language MNIST and ASL Alphabet. These studies showed that hand localisation and isolated letter classification were tractable, but they did not address the temporal structure required for word-level ASL recognition. Their role in the final method is therefore diagnostic: they justify the shift from pixel-based letter recognition to landmark-based word recognition.

4.2.5 Proof-of-Concept Design and Expected Outcomes

The proof of concept connects the trained recognition model to a runtime application. The mobile client captures a short video clip and sends it to a FastAPI server. The server applies the same MediaPipe landmark extraction used during training, loads the selected checkpoint and returns Top-K predictions with confidence scores. The client can display the prediction shortlist and optionally convert the Top-1 label to speech.

The expected outcome of the proof of concept is not a deployment-ready translator. Its purpose is to demonstrate that the trained landmark-based recogniser can be integrated into an end-to-end capture, inference and response loop. The quantitative validity of the recognition model is therefore assessed in the Experiments chapter, while the prototype demonstrates the feasibility of using the selected model in an interactive setting.

4.2.6 Replication Support

Replication depends on controlled configuration and traceable experiment outputs. Dataset paths, model options and training settings are stored in YAML configuration files under `configs/`. Each training or evaluation run writes an `experiment_manifest.json` file with the git commit, resolved configuration, hardware identifier, wall-clock time and final metrics. The development environment is specified through `pyproject.toml` for the training package and `04_server/requirements.txt` for the inference server. The full reproduction procedure is documented in `docs/reproducibility_guide.md`.

4.3 Methodology

The project followed a Lean Startup-inspired iterative methodology. In this work, the term does not refer to commercial product validation. It refers to short research and development cycles in which a technical assumption is built, measured and revised before the next cycle begins. This approach was appropriate because the early project direction changed after the exploratory stage, and the work plan had to preserve room for evidence-driven correction.

Each iteration followed four stages:

1. **Hypothesis definition.** The input was a previous devlog entry, tutor comment, experimental result or implementation problem. The output was a testable technical hypothesis, together with the dataset, metric and configuration needed to evaluate it.
2. **Build.** The input was the hypothesis and the current repository state. The output was an implementation change, a YAML configuration file or both, committed to Git so that the experiment could be traced to a specific version of the code.
3. **Measure.** The input was the committed implementation and its configuration. The output was a set of experiment artefacts: metrics, checkpoints, plots and an `experiment_manifest.json` file.
4. **Learn.** The input was the measured evidence from the previous stage. The output was a documented decision to keep, revise or discard the hypothesis, recorded in the development log before the next iteration began.

The pivot recorded in Section 6.2 is the clearest application of this loop. The project began with a pixel-based route that combined hand localisation with static letter classification. The detector and the letter classifiers were functional on their own datasets, but the combined route showed that the central problem was not locating the hand. The central problem was representing the temporal structure of complete signs. The next iteration therefore replaced the pixel-based recognition route with the landmark-based word-recognition pipeline used in the rest of the project.

Tutor meetings acted as external review checkpoints within the iterative process. Examples include removing Test-Time Augmentation (TTA) from the final evaluation protocol, adopting Stratified 5-Fold Cross-Validation, extending validation to MS-ASL50 and adding multi-seed evaluation to reduce dependence on a single random initialisation.

Documentation was maintained as part of the methodology rather than as a final clean-up task. A structured development log was kept throughout the project, containing dated entries for each main work session and recording the hypothesis, change, result and next action of each iteration. The work plan in Section 4.4 is reconstructed from that log, the proposal, experiment artefacts and supervision notes.

4.4 Work Plan

The executed work plan was reconstructed from the original proposal, development notes, experiment artefacts and supervision notes. It is organised into seven phases (A–G): the first two correspond closely to the proposal-era plan, while the remaining phases reflect the methodological pivot from static, pixel-based recognition to landmark-based word-level ASL recognition.

The original work plan from the project proposal (January 2026) followed a sequential structure with separate phases for hand detection, static handshape recognition, dynamic sign recognition, temporal fusion, language generation, mobile prototype, integration and validation. The executed plan diverged after the methodological pivot described in Section 6.2, which replaced the static letter recognition route with landmark-based word recognition and folded the temporal fusion stage into the selected sequence model.

Figure 4.1 summarises the executed work plan at the start of the section. The chart shows the start and end of each phase and makes the main deviation from the initial proposal visible: the shift from static letter recognition to landmark-based word-level recognition.

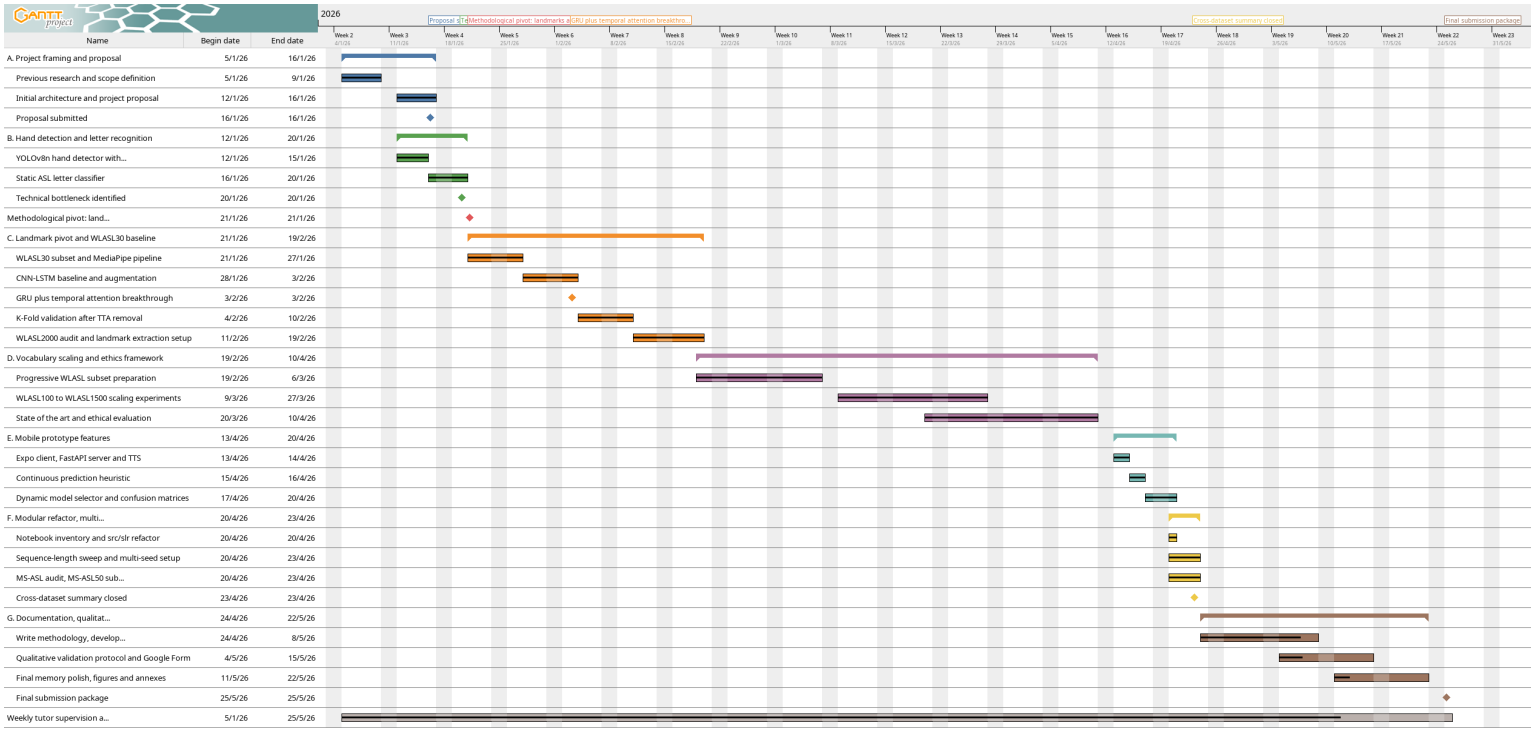


Figure 4.1: Project Gantt chart showing the executed work plan grouped into seven phases, from project framing and initial hand-detection experiments to landmark-based word recognition, cross-dataset validation, prototype integration and final documentation. Source: self-created with GanttProject.

Phase A: Project framing and proposal (5 January 2026 to 16 January 2026). The project formally started with problem definition, preliminary literature review, scope selection and proposal preparation. This phase defined the accessibility motivation, the initial system architecture, the first work schedule and the research direction. No model training was carried out at this stage. The main output was the approved project proposal and its initial planning material.

Phase B: Hand detection and static letter recognition (12 January 2026 to 20 January 2026). The initial technical work explored the proposal's first recognition route: localising hands in images and classifying static ASL letters. A YOLOv8n detector was trained on a combined EgoHands and Kaggle Hand Dataset corpus, and CNN-based classifiers were explored on static ASL alphabet datasets. These components reached working performance on their own tasks, but they also revealed a limitation in the original direction: the central research challenge was not hand localisation, but representing the temporal structure of complete signs.

Phase C: Landmark pivot and WLASL30 baseline (21 January 2026 to 19 February 2026). The project pivoted from pixel-based letter recognition to word-level recognition using MediaPipe hand landmarks. WLASL30 was built as the first controlled benchmark, and several sequence architectures were compared under the same landmark input representation. The GRU+Attention architecture became the best-performing configuration because it combined recurrent temporal modelling with an attention layer able to weight informative frames. After supervisory review, TTA was excluded from the final evaluation protocol, and the WLASL30 result was strengthened through Stratified 5-Fold Cross-Validation.

Phase D: Vocabulary scaling and ethical framework (19 February 2026 to 10 April 2026). Once the GRU+Attention model had been selected, the next phase tested how the approach behaved as the WLASL vocabulary increased beyond the initial 30-class setting. The model was retrained on progressively larger WLASL subsets to quantify the effect of vocabulary size and per-class data scarcity. In parallel, the ethical and social-impact chapter was developed, including the accessibility motivation, privacy implications of landmark-based representations, bias considerations, reproducibility and the responsible-deployment limits of the prototype.

Phase E: Runtime prototype (13 April 2026 to 20 April 2026). The technical work expanded from offline training to a runtime proof of concept. A mobile client and FastAPI server were integrated so that short clips could be captured, sent for inference and returned as Top-K word predictions. Additional prototype features included text-to-speech, model selection from the client and a heuristic continuous-prediction demo mode based on overlapping windows. This phase demonstrates end-to-end feasibility, but it is not treated as the main research result. The quantitative contribution remains the landmark-based recognition pipeline and its validation.

Phase F: Refactoring and cross-dataset validation (20 April 2026 to 23 April 2026). The notebook-based experimental code was consolidated into a modular Python package, with configuration-driven training, structured experiment manifests, reusable evaluation scripts and preserved checkpoints and plots. This refactor improved reproducibility and made it possible to rerun the same pipeline across datasets. MS-ASL50 was then added as an independent validation benchmark, and the final evaluation protocol incorporated multi-seed runs, a sequence-length analysis, cross-dataset validation and comparison against the WLASL results.

Phase G: Documentation and qualitative validation (24 April 2026 to 25 May 2026). The final phase focused on document writing, result consolidation, figure and table preparation and qualitative validation of the runtime prototype. The experimental chapter was organised into quantitative and qualitative tracks. The quantitative track reports completed model evaluations, while the qualitative track describes the participant protocol and reports the usability evidence collected from the participant sessions.

4.5 Constraints and Limitations

This section lists the constraints that shaped the work plan and implementation. They are ordered by their impact on project scope and design. Limitations of the experimental conclusions themselves, including standard deviations, test-set granularity and single-architecture cross-dataset validation, are treated separately in Section 6.8.

1. **Computing capacity.** Every training and evaluation run executed on a single NVIDIA RTX 3050 Ti Laptop GPU with 4 GB of VRAM. This imposed a hard memory ceiling on the temporal window, batch size and model size that could be tested. The practical effect was to prioritise compact recurrent architectures and to exclude larger video models that would not fit reliably within the available hardware.
2. **Dataset reconstruction and landmark coverage.** Both word-recognition datasets depend on external video sources rather than on locally distributed video files. WLASL relies on URLs that have decayed since publication, and the MediaPipe detection rate on the surviving WLASL clips is approximately 69 %. MS-ASL is reconstructed from per-instance YouTube URLs, and the reconstruction audit records a download success rate of 65.9 % (16,816 of 25,513 clips). This constrained the effective number of usable training examples and determined the feasible subset definitions, including the use of MS-ASL50 instead of the more standard MS-ASL30 baseline used in WLASL-style comparisons, since only six MS-ASL classes met the train/val/test sample threshold needed for an MS-ASL30 reproduction (cf. Section 6.6).
3. **Experimental calendar.** The executed work plan ran from 5 January 2026 to the final submission window in May 2026. This fixed calendar limited how many repeated experiments could be completed. Multi-seed evaluation was applied to the main reported configurations, but broader vocabulary-scaling runs and additional cross-dataset architecture comparisons were not included in the experimental scope.
4. **Linguistic scope.** The project validates a single sign language, American Sign Language, and uses ASL exclusively for both the WLASL and MS-ASL benchmarks. The pipeline contains no ASL-specific rule-based component, and each component that consumes landmarks could in principle be retrained on a different sign language given a comparable dataset. That validation has not been performed, so transfer to Spanish Sign Language, British Sign Language or any other sign language remains future work.

Chapter 5

Technical Development

This chapter describes the research and engineering foundations of this FDP: how the recognition solution is structured into independent stages, the methodological investigations that informed the choice of algorithms and hyperparameters, and the functional capabilities provided by each stage. It also details the execution environment and the requirements that drove the architectural design. Finally, it describes the experimental datasets that feed the recognition pipeline.

5.1 Solution design

The solution is structured as a modular three-stage architecture, integrating research-driven methods across data preparation, model training and evaluation, and runtime inference. The stages are physically separated (different processes and different machines in the execution environment) but share a single source of truth for data formats and trained model artefacts, so a checkpoint produced during model training can be served unchanged during runtime inference. Figure 5.1 shows the solution architecture and its execution flow.

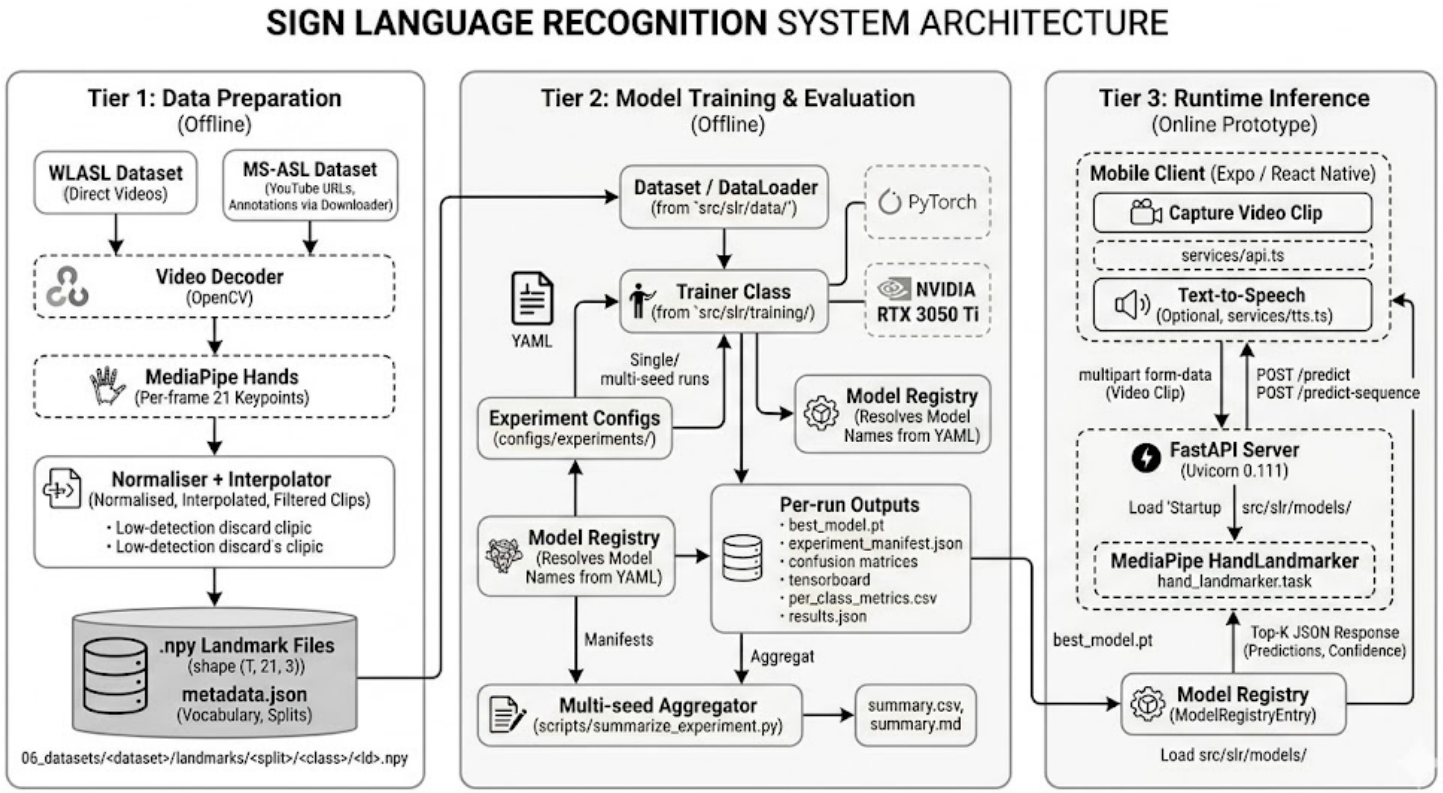


Figure 5.1: Solution architecture showing the data preparation, training and evaluation, and runtime inference stages. Source: self-created from the implemented pipeline.

5.1.1 Stage 1: Data preparation

The data preparation stage converts raw benchmark videos into landmark sequences that can be used by the training and inference stages. It corresponds to the first two stages of the recognition pipeline introduced in Section 4.2.2, namely video acquisition and decoding, and landmark extraction and normalisation. This section decomposes those stages into five execution phases:

1. **Video acquisition.** The input is the official Word-Level American Sign Language (WLASL) distribution or the Microsoft American Sign Language (MS-ASL) annotation files. WLASL clips are downloaded from the dataset distribution, while MS-ASL clips are reconstructed from the referenced YouTube videos. The output is a local set of labelled video clips aligned with their gloss and split metadata.
2. **Clip decoding and temporal trimming.** OpenCV decodes each video frame by frame. For MS-ASL, each source video is trimmed with the per-instance `(start_time, end_time)` pair. The output is an ordered frame sequence for one sign instance.
3. **Hand landmark extraction.** MediaPipe Hands estimates 21 three-dimensional landmarks per valid frame. The output for each frame is a structured hand representation with coordinates for the wrist, finger joints and fingertips.
4. **Geometric normalisation and missing-frame handling.** Each frame is wrist-centred and length-normalised by the wrist-to-middle-finger distance. Frames where MediaPipe fails to detect a hand are linearly interpolated from neighbouring valid frames, and clips whose detection rate falls below 0.3 are discarded.
5. **Serialisation.** Each surviving clip is stored as a per-clip `.npy` array of shape $(T, 21, 3)$. The accompanying `metadata.json` records the dataset name, gloss vocabulary, split assignments and source-video identifiers.

This stage is implemented in `src/slr/data/` and exposed through the `scripts/extract_landmarks_*.py` entry points.

5.1.2 Stage 2: Model training and evaluation

The training and evaluation stage turns landmark sequences into reproducible model comparisons. Its execution follows four phases:

1. **Configuration resolution.** The input is a YAML configuration under `configs/`. The file specifies the dataset path, model identifier, optimiser, sequence length and evaluation settings. Shared defaults are declared in `configs/base/gru_attention.yaml`, and experiment-specific files override only the fields that change.
2. **Model construction.** The registry in `src/slr/models/` resolves the model identifier into one of the implemented sequence architectures. The supported identifiers are `gru_baseline`, `gru_attention` and `transformer`.
3. **Optimisation and evaluation.** The trainer in `training/trainer.py` optimises the selected architecture and evaluates it through the shared evaluator in `evaluation/`. The same trainer supports the multi-seed protocol used to report mean and standard deviation in the experiments chapter.

4. **Artefact generation.** Each run produces a self-contained directory with the best checkpoint by validation Top-1, the training history, confusion matrices, per-class metrics and an `experiment_manifest.json` recording git commit, hardware, runtime, hyperparameters and final metrics.

The experiment-tracking code is implemented in `tracking/manifest.py` and `tracking/git_utils.py`.

5.1.3 Stage 3: Runtime inference

The runtime inference stage integrates the trained recogniser into a mobile/server proof of concept. Its execution follows five phases:

1. **Video capture.** The React Native and Expo client in `03_app/signtotext-expo/` records a short video clip with the device camera.
2. **Request submission.** The client sends the clip to the FastAPI server in `04_server/main.py` through the REST client implemented in `services/api.ts`.
3. **Server-side preprocessing.** The server applies the same MediaPipe landmark extraction, normalisation and sequence preparation used in the data preparation stage.
4. **Prediction.** The server loads the requested checkpoint from the model registry and returns Top-K predictions with confidence scores through `POST /predict`. The `POST /predict-sequence` endpoint provides a heuristic continuous-sequence variant.
5. **Client response.** The client displays the prediction shortlist and can pass the Top-1 label to the text-to-speech service in `services/tts.ts`.

The server reuses the `src/slr/` package directly, so the inference code path remains aligned with the evaluation code path.

5.1.4 Description of functional features

The solution exposes six high-level capabilities. Each one is justified by the technical decision it embodies, so this list doubles as a record of why the architecture took its current shape.

- **F1: Landmark-based input pipeline.** Each frame is reduced from a 224×224 RGB tensor to 63 floating-point landmark coordinates, a roughly 2,389$\times$ dimensionality reduction. The choice removes background and skin-tone signal at source, which both shrinks the model and disconnects recognition accuracy from appearance bias.
- **F2: Multi-seed training protocol.** The trainer supports running the same configuration across multiple random seeds and aggregating the metrics with mean and standard deviation. Without this protocol, a single-seed Top-1 figure on a small test set is indistinguishable from noise; with it, a 2–3 percentage-point standard deviation can be reported alongside every headline number.
- **F3: Architecture-agnostic training loop.** The same `Trainer` accepts three model identifiers through a registry lookup: `gru_baseline` is a bidirectional GRU architecture without attention, `gru_attention` is the selected GRU architecture with a temporal attention layer, and `transformer` is a self-attention architecture used as a comparison baseline. This makes the architecture comparison reproducible: only the model field of the YAML changes between runs, every other variable stays fixed.

- **F4: Cross-dataset validation.** The same pipeline runs unmodified on WLASL and on MS-ASL. Only the dataset path and the gloss vocabulary differ between the two configurations; nothing in the model, optimiser or loss is dataset-specific, which is what makes the cross-dataset comparison fair.
- **F5: Reproducible experiment tracking.** Every run writes an `experiment_manifest.json` that captures git commit, dataset metadata, hyperparameters, hardware, runtime, and final metrics in one place. A future user can reconstruct the exact configuration of any historical run from this single file.
- **F6: Top-K predictions in inference.** The runtime API returns Top-1 and Top-5 candidates with confidence scores instead of forcing a single answer. This matches the way the model itself is evaluated and lets the client surface a shortlist when the Top-1 confidence is low.

For example, the `experiment_manifest.json` generated for the MS-ASL50 seed 2026 run records the dataset identifier, random seed, split sizes, GRU+Attention configuration, hardware, runtime, checkpoint path and final metrics, including 71.19% Top-1 and 91.53% Top-5 accuracy. This is the concrete output produced by the reproducible experiment-tracking feature.

5.1.5 Operational Environment

Training and landmark extraction were performed in the local development environment; the runtime prototype was tested with a paired mobile client and an HTTP server. Table 5.1 summarises the hardware, software runtime and storage formats under which the results in this FDP were produced.

Table 5.1: Operational environment for training, landmark extraction, and runtime inference.

Environment area	**Specification**
Operating system	Ubuntu 24.04 LTS (training and server); cross-platform (mobile client).
CPU	Multi-core x86_64 (local laptop).
GPU	NVIDIA RTX 3050 Ti Laptop, 4 GB VRAM, CUDA 12.x.
Python	3.10 (project); 3.11 supported via `requires-python = ">=3.10"`.
Deep learning framework	PyTorch 2.x with CUDA support.
Pose estimation	MediaPipe Hands 0.10+ (`HandLandmarker`).
Video processing	OpenCV 4.8+.
Mobile client framework	React Native + Expo.
Backend	FastAPI 0.111 with Uvicorn 0.29.
Persistence	JSON manifests, `.npy` landmark files, `.pt` PyTorch checkpoints.
Version control	Git.

The detailed installation and execution procedure is not repeated in this section. It belongs to the user manual appendix, where the environment setup, dataset preparation, training commands, evaluation commands and server execution procedure are described together.

5.2 Requirements

5.2.1 Description of requirement attributes

For the textual description of the different identified requirements, a set of attributes has been selected to describe each requirement. Below is a description of the meaning of each attribute used:

- **Code**: A unique abbreviated identifier for the requirement. It is constructed using the requirement type code followed by a hyphen and three digits. Requirements are divided into functional and non-functional requirements. The codes used are **FR** for Functional Requirements and **NFR** for Non-Functional Requirements. For example: `FR-001`.
- **Name**: Extended identification of the requirement.
- **Description**: A basic description of the identified requirement.
- **Source**: Indicates the source through which the requirement was identified. This value usually corresponds to one or more use case codes.
- **Necessity**: Determines the degree of implementation required for the requirement. The possible values are:
  - **Essential**: The requirement must be implemented.
  - **Desirable**: It is preferable to implement the requirement, but it is not mandatory.
  - **Optional**: The requirement may be implemented, but it is neither important nor mandatory.
- **Priority**: Defines the importance of the requirement, allowing the order in which requirements are included in the design and implementation process to be established. The possible values are:
  - **High**: The requirement must be implemented in the early stages of development.
  - **Medium**: The requirement should be implemented once high-priority requirements have been completed.
  - **Low**: The requirement should be implemented in the final stages of development. These requirements do not affect the correct functioning of the system.
- **Stability**: Defines the stability of the requirement throughout the software life cycle. This indicates whether the requirement may be modified during the life cycle. The possible values are:
  - **Stable**: The requirement will not change during the system life cycle.
  - **Unstable**: The requirement may change during the system life cycle.
- **Verifiability**: Defines the degree to which a requirement can be verified, that is, the extent to which it is possible to confirm that the requirement has been implemented in the developed system. The possible values are:
  - **High**: It is possible to verify that the requirement has been implemented. These requirements usually correspond to core system functionalities.

- **Medium**: It is possible to verify that the requirement has been implemented, but it requires complex testing or inspection of the source code.
- **Low**: It is difficult, or in some cases impossible, to verify whether the requirement has been implemented.

5.2.2 Functional Requirements

The system specifies four functional requirements covering the core recognition pipeline (FR-001, FR-002), the experiment-tracking infrastructure (FR-003), and statistical validation through multi-seed training (FR-004).

System requirement			
ID	FR-001	**Source**	Pipeline design
Name	Landmark extraction from video clips		
Description	The system shall extract 21 three-dimensional hand landmarks per frame from each input video clip using MediaPipe Hands, producing a per-clip array of shape $(T, 21, 3)$ stored as a `.npy` file alongside dataset metadata.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.2: FR-001: Landmark extraction from video clips

System requirement			
ID	FR-002	**Source**	Project objective
Name	Word-level sign classification		
Description	The system shall classify input video clips into one of the N predefined ASL word classes for the active dataset, returning Top-1 and Top-5 predictions accompanied by confidence scores.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.3: FR-002: Word-level sign classification

System requirement			
ID	FR-003	**Source**	Reproducibility requirements
Name	Reproducible experiment tracking		
Description	Every training run shall generate a structured manifest capturing dataset version, hyperparameters, git commit, hardware, runtime, and final metrics in a single JSON file, sufficient to reconstruct the run configuration without consulting external sources.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.4: FR-003: Reproducible experiment tracking

System requirement			
ID	FR-004	**Source**	Statistical validation
Name	Multi-seed training support		
Description	The training subsystem shall support running the same configuration across multiple random seeds and aggregating the resulting metrics with mean and standard deviation, so that every reported result can be accompanied by an uncertainty estimate.		
Necessity	Essential	**Priority**	Medium
Stability	Stable	**Verifiability**	High

Table 5.5: FR-004: Multi-seed training support

5.2.3 Non-functional Requirements

Three non-functional requirements characterise quality attributes of the system: runtime performance (NFR-001), code quality and modularity (NFR-002), and documentation completeness (NFR-003).

System requirement			
ID	NFR-001	**Source**	User experience
Name	Inference latency		
Description	End-to-end inference for a single video clip shall complete in under two seconds on the target deployment hardware (mobile client recording the clip, server running landmark extraction and model prediction).		
Necessity	Desirable	**Priority**	Medium
Stability	Unstable	**Verifiability**	Medium

Table 5.6: NFR-001: Inference latency

System requirement			
ID	NFR-002	**Source**	Maintainability
Name	Modular code architecture		
Description	The codebase shall be organised as a Python package with separate submodules for data, models, training, evaluation, tracking, and utilities, supporting independent unit testing of each submodule.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.7: NFR-002: Modular code architecture

System requirement			
ID	NFR-003	**Source**	Academic rigor
Name	Documentation completeness		
Description	The repository shall include a reproducibility guide that enables a third party to replicate the headline experimental results with documented commands and dependency declarations.		
Necessity	Essential	**Priority**	Medium
Stability	Stable	**Verifiability**	Medium

Table 5.8: NFR-003: Documentation completeness

5.3 Datasets

This FDP spans two recognition tasks with different data requirements: hand localisation from still images and word-level sign language recognition from video. Six datasets are therefore used across the experimental pipeline. This section formalises the properties of each dataset by first defining a common set of descriptive attributes, then presenting a comparative table and finally describing each dataset with the same structure: metadata and technical specifications, composition and volume, source, and preparation.

5.3.1 Dataset Attribute Definitions

To ensure a consistent and unambiguous characterisation of every dataset, the following attributes are used in Table 5.9. Each attribute captures a distinct dimension relevant to dataset selection, reproducibility, and the interpretation of experimental results:

- **ID**: A unique alphanumeric identifier assigned to each dataset within this FDP. The prefix `DS-` followed by a two-digit index allows unambiguous cross-referencing in subsequent chapters when reporting experimental configurations and results.
- **Name**: The official or widely-used name of the dataset as it appears in the literature or on its hosting platform.
- **Year**: The year of original publication or first public release. This contextualises the dataset within the chronological development of the field and helps evaluate whether it reflects current data collection practice.
- **Task**: The specific sub-task for which the dataset was designed. Three task categories appear in this FDP:
  - **Hand Detection**: the dataset provides annotated bounding boxes or segmentation masks marking the location of hands in images.
  - **Letter Recognition**: the dataset provides labelled static images of individual American Sign Language (ASL) letter signs.
  - **Word Recognition**: the dataset provides labelled video clips of complete ASL word signs, requiring temporal modelling.
- **Samples**: The total number of annotated data instances. For image datasets, this equals the number of images; for video datasets, it equals the number of individual video clips. Where a

subset of the full dataset was used in this FDP, this is noted in the individual descriptions.

- **Classes**: The number of distinct output categories. For hand detection datasets this is always 1 (the single *hand* class); for sign language datasets it corresponds to vocabulary size, meaning the number of distinct letters or word glosses.
- **Format**: The storage format and annotation type of the raw data. This attribute specifies whether instances are static images or video clips, the image resolution where relevant, and the form of ground-truth annotation, such as bounding box coordinates in XML, pixel-level segmentation masks, image class labels or video-level class labels.
- **Input Representation**: Whether the models trained on this dataset consume *pixel-based* data (raw RGB or greyscale image/video frames) or *landmark-based* data (structured three-dimensional keypoint coordinates extracted by a pose estimator such as MediaPipe). Pixel-based inputs preserve full visual detail but are high-dimensional and sensitive to background, lighting and signer appearance. Landmark-based inputs are compact and less exposed to visual variability, at the cost of depending on a reliable keypoint extractor.

5.3.2 Comparative Overview

Table 5.9 provides a high-level comparison of the six datasets used in this FDP. Datasets DS-01 and DS-02 support the hand localisation pipeline. Datasets DS-03 and DS-04 were used in the initial letter-level classification phase, which was later superseded by word-level recognition. Datasets DS-05 and DS-06 are the video benchmarks used for the final landmark-based ASL recognition pipeline.

Table 5.9: Comparative overview of all datasets used in this FDP.

ID	Name	Year	Task	Samples	Classes	Format
DS-01	EgoHands	2015	Hand Detection	4,800	1	Images + seg. masks
DS-02	Kaggle Hand Dataset	2023	Hand Detection	∼4,200	1	Images + BBox (XML)
DS-03	Sign Language MNIST	2017	Letter Recog.	34,627	24	28×28 greyscale CSV
DS-04	ASL Alphabet	2018	Letter Recog.	87,000	29	200×200 RGB JPG
DS-05	WLASL	2020	Word Recog.	21,083	2,000	RGB video clips
DS-06	MS-ASL	2018	Word Recog.	25,513	1,000	RGB video clips, YouTube

5.3.3 Hand Detection Datasets

Datasets DS-01 and DS-02 are used jointly to train the hand localisation module based on You Only Look Once version 8 nano (YOLOv8n). Combining the two sources increases viewpoint diversity: an egocentric-only detector may fail when the camera is positioned at eye level, while a detector trained only on third-person images may struggle with first-person perspectives. After merging, duplicate removal and a stratified train/validation/test split, the combined training set comprises approximately 5,223 annotated images.

DS-01: EgoHands

Metadata and technical specifications. EgoHands, introduced by Bambach et al. [28], is an egocentric hand localisation dataset published in 2015. Ground-truth annotations are stored as pixel-

level segmentation polygons in MATLAB `.mat` files. The input representation used in this FDP is pixel-based.

Composition and volume. The dataset contains approximately 4,800 labelled frames extracted from 48 video recordings. Most frames contain two visible hands and include overlap or occlusion cases during everyday interactions such as card games, chess and jigsaw puzzles.

Source. Indiana University collected the videos using a GoPro camera mounted on the chest of an observer watching another person. This first-person perspective makes the dataset useful for close-range hand detection but limits its viewpoint diversity.

Preparation. The segmentation polygons were converted to axis-aligned bounding boxes in YOLO format. EgoHands was then merged with DS-02 to reduce the first-person bias of the training corpus.

DS-02: Kaggle Hand Dataset

Metadata and technical specifications. The Kaggle Hand Dataset is a collection of annotated hand images published on Kaggle [29]. Ground-truth bounding boxes are provided in Pascal Visual Object Classes (VOC) XML format. The input representation used in this FDP is pixel-based.

Composition and volume. The dataset contains approximately 4,200 labelled images of hands in third-person perspectives. The images include varied backgrounds, lighting conditions, skin tones and partial occlusions.

Source. The dataset is distributed through the Kaggle platform and complements DS-01 by adding images that are not captured from the camera wearer's viewpoint.

Preparation. Bounding-box coordinates were normalised to the YOLO format used by the detection pipeline. The joint corpus formed by DS-01 and DS-02 yielded a YOLOv8n model with approximately 98% mean average precision at 50% overlap (mAP50) and 87% mean Intersection over Union (Mean IoU) on the held-out test split.

5.3.4 Letter Recognition Datasets

Datasets DS-03 and DS-04 were used during the initial phase of the FDP, when the recognition target was the static letter-level ASL alphabet. Both datasets cover 24 of the 26 ASL letters; the letters *J* and *Z* are excluded in both cases because they require hand motion and cannot be represented by a single static image. The two sources were merged and unified into a single training corpus for letter classification experiments. Although this phase was later superseded by the word-level recognition approach, these datasets remain relevant because they established the viability of data-driven letter classification and informed the design of the preprocessing pipeline. The input representation for both datasets is pixel-based.

DS-03: Sign Language MNIST

Metadata and technical specifications. Sign Language MNIST, hosted on Kaggle [30], mirrors the format of the MNIST handwritten digit benchmark. Each sample is a 28×28 greyscale image distributed in CSV format, with one label column and one pixel-intensity column per image position. The input representation is pixel-based.

Composition and volume. The dataset contains 27,455 training images and 7,172 test images, for a total of 34,627 samples. Each sample represents one of 24 static ASL letters.

Source. The dataset was constructed from ASL alphabet photographs that were rescaled and converted to the MNIST-style format.

Preparation. The low resolution and greyscale format made the dataset simple to load and normalise, but it also removed fine-grained finger articulation details. For the initial letter-recognition experiments, the dataset was restricted to the 24 classes shared with DS-04.

DS-04: ASL Alphabet

Metadata and technical specifications. ASL Alphabet, distributed on Kaggle [31], provides RGB images at a resolution of 200×200 pixels. The input representation is pixel-based.

Composition and volume. The dataset contains approximately 87,000 images organised into 29 classes: the 24 static ASL letters, the *space* class and two non-letter classes (*del*, *nothing*). Each class has 3,000 training images captured in a controlled environment.

Source. The images were collected for static ASL alphabet recognition and provide a higher-resolution complement to DS-03.

Preparation. For integration with DS-03, images were converted to greyscale and resized to a common format. The unified corpus contained approximately 79,000 training images and 7,000 test images covering the 24 shared letter classes.

5.3.5 Word Recognition Datasets

Datasets DS-05 and DS-06 are the primary benchmarks for word-level ASL recognition. WLASL is the dataset on which the architecture was tuned, while MS-ASL was added later to validate the same pipeline on an independent ASL corpus with a different signer distribution.

DS-05: WLASL

Metadata and technical specifications. The Word-Level American Sign Language (WLASL) dataset, published by Li et al. [1], contains 21,083 video instances covering 2,000 ASL glosses. Each instance is a short RGB video labelled with a single word-level gloss.

Composition and volume. Each clip shows a single signer performing one complete sign, typically against a plain or low-clutter background, with durations ranging from approximately one to four seconds. WLASL is partitioned into progressively larger evaluation subsets: WLASL100, WLASL300, WLASL1000 and WLASL2000.

Source. Videos were collected from multiple online platforms, including YouTube channels maintained by ASL educators, online courses and signing dictionaries. Qualified ASL experts manually verified the sign production. A practical limitation is that WLASL relies on external video URLs, so some clips become unavailable over time.

Preparation. Raw RGB frames are not used as model input in this FDP. MediaPipe Hands [8] is applied to each frame to extract 21 three-dimensional hand landmarks, yielding 63 scalar features per frame. The effective detection rate of MediaPipe on WLASL clips is approximately 69%; frames where no hand is detected are linearly interpolated from adjacent detected frames, and each video is resampled to a fixed sequence length of 64 frames.

For the experiments described in this FDP, the following WLASL subsets were extracted and used:

- **WLASL30**: The 30 most frequent glosses, used for initial architecture exploration and model selection. Split: 242 training / 56 validation / 34 test videos.
- **WLASL100**: The 100 most frequent glosses, used for K-Fold cross-validation and multi-split evaluation. Split: 741 training / 162 validation / 99 test videos.
- **WLASL300, WLASL1000, WLASL2000**: Larger subsets targeted for progressive scaling experiments, where the goal is to assess model accuracy and generalisation at increasing vocabulary sizes.

DS-06: MS-ASL

Metadata and technical specifications. The Microsoft American Sign Language (MS-ASL) dataset, published by Vaezi Joze and Koller [2], defines 25,513 video instances over 1,000 glosses. The original release provides annotation JSON files rather than distributing the video clips directly.

Composition and volume. The official split contains 16,054 training samples, 5,287 validation samples and 4,172 test samples. The vocabulary is organised into four cumulative subsets (MS-ASL100, MS-ASL200, MS-ASL500 and MS-ASL1000) obtained by filtering on the integer label.

Source. Each annotation references a YouTube URL, a per-instance `(start_time, end_time)` interval and a normalised signer bounding box. Of the 25,513 declared instances, **16,816 were successfully downloaded (65.9% coverage)**, with split-level coverage of 74.0% / 47.5% / 58.0% for train, validation and test. The remaining clips were unavailable because the source videos had been removed or made private.

Preparation. A disk-level audit graded every downloaded clip by integrity and training viability. Only **6** of the top-30 classes met the strict MS-ASL30 threshold of 30 train / 5 validation / 5 test surviving samples, while **17** met the MS-ASL50 threshold of 20 / 5 / 5 and **57** met the MS-ASL100 threshold of 20 / 3 / 3. The chosen subset is therefore **MS-ASL50**, defined as the 50 classes with the largest training counts that satisfy the 20 / 3 / 3 threshold. The subset was processed with the

same MediaPipe pipeline used for WLASL: each frame was reduced to 21 three-dimensional hand landmarks and stored as a per-clip `.npy` array. After extraction, the final per-split counts are **1,410 train / 306 validation / 236 test (1,952 total)**, with 31 source clips dropped because their mean detection rate fell below 0.3. The mean detection rate across the surviving clips is **83.9%**.

Chapter 6

Experiments

This chapter presents the experimental process chronologically, reflecting the project's evolution from its inception to final validation. It begins with the initial hand-detection and letter-classification pilot studies that informed the subsequent methodological pivot. The work then progressed into a rigorous architectural benchmarking phase and an extensive hyperparameter sweep, which ultimately yielded the best-performing configuration. Finally, the process concluded with multi-seed, cross-validation and cross-dataset validation to assess statistical stability. Every result reported below is backed by stored experiment artefacts, including configurations, metrics, plots, trained checkpoints and an experiment manifest for each run. The detailed repository paths needed to reproduce or audit the results are centralised in the reproducibility guide and internal audit material, so the chapter cites experiment outputs conceptually rather than repeating long file paths in every caption.

6.1 Experimental Methodology

The experimental methodology is organised as a sequence of phases. Each phase has a defined input, configuration, measurement procedure and output, so that later sections can report results without mixing datasets, hyperparameters and interpretation in the same paragraph. The comparative metrics used throughout the chapter are those defined in Section 4.2: Top-1 accuracy, Top-5 accuracy, Macro-F1 and balanced accuracy.

M1: Exploratory pilots. **Input:** EgoHands, Kaggle Hand Dataset, Sign Language MNIST and ASL Alphabet. **Configuration and measurements:** YOLOv8n, SSDLite-MobileNetV3 and EfficientDet-D0 were compared for hand localisation, while CNN classifiers were trained for static ASL letters. Detection used mAP50, Mean IoU and latency; letter recognition used classification accuracy. **Output:** evidence that detection and static letters were tractable but insufficient for word-level ASL recognition.

M2: Architecture selection. **Input:** WLASL30 landmark sequences, with 242 train, 56 validation and 34 test clips. **Configuration and measurements:** five locally tuned architecture families were compared under the same split, preprocessing and hardware. Each run was evaluated on the fixed

test split. **Output:** selection of the best-performing GRU+Attention configuration for downstream experiments.

M3: Sequence-length sweep. Input: WLASL30 landmark sequences and the selected GRU+Attention architecture. **Configuration and measurements:** three maximum sequence lengths were tested, 32, 64 and 96 frames, each with seeds 42, 123 and 2026. **Output:** selection of `max_frames`=64 as the default temporal input length.

M4: WLASL30 validation. Input: the WLASL30 fixed split and WLASL100 filtered to the same 30 glosses. **Configuration and measurements:** the fixed split used three seeds, while the complementary validation used Stratified 5-Fold Cross-Validation with class distribution preserved. **Output:** mean and standard deviation estimates for the selected configuration under seed and split variation.

M5: Cross-dataset validation. Input: MS-ASL50 landmark sequences, with 1,410 train, 306 validation and 236 test clips. **Configuration and measurements:** the selected configuration was retrained with the same preprocessing and three seeds, changing only the dataset path and vocabulary. **Output:** evidence about whether the pipeline reproduced on a second ASL corpus.

M6: Vocabulary scaling. Input: WLASL subsets from 100 to 1,500 classes, plus the 30-class reference. **Configuration and measurements:** the selected configuration was retrained with `max_frames`=64 and seed 42. Larger subsets used the extended budgets recorded in their manifests. **Output:** a scaling curve relating vocabulary size, per-class data availability and test performance.

Unless a phase states otherwise, training and evaluation execute on the local workstation described in Section 5.1.5: NVIDIA RTX 3050 Ti Laptop GPU (4 GB VRAM, CUDA 12.x). The standard optimisation schedule uses AdamW with learning rate 1×10^{-3}, weight decay 5×10^{-4} and label smoothing of 0.1 on the cross-entropy loss. The learning rate follows a CosineAnnealingLR schedule. Early stopping monitors validation Top-1 with patience 30 epochs, and the maximum standard run length is 150 epochs.

Two reliability protocols are used when the sample size permits it. The **multi-seed protocol** repeats the same configuration with seeds 42, 123 and 2026, then reports mean $\pm$ standard deviation. This is required on WLASL30 because the test split contains only 34 samples, so one error changes Top-1 by approximately 2.94 percentage points. The **Stratified 5-Fold Cross-Validation** protocol uses WLASL100 filtered to the same 30 classes, 332 total samples and five non-overlapping folds with class distribution preserved. It complements the multi-seed protocol by varying the test split itself rather than only the model initialisation.

Test-Time Augmentation (TTA) was excluded from the evaluation protocol. Earlier exploratory runs on WLASL30 showed that TTA increased the reported Top-1 without a corresponding gain in true generalisation. Every number in this chapter therefore reflects a single forward pass per test sample.

Part I: Quantitative Evaluation

6.2 Phase 1: Exploratory Work and Methodological Pivot

The first months of the project explored two avenues that were eventually superseded by the landmark + word approach. They are reported here because the lessons drawn from them are the technical justification of the pivot, not because they appear as headline numbers in the final system. This section applies Phase M1 from Section 6.1: the input is the set of hand-detection and letter-recognition datasets, the output is the decision to abandon pixel-based pilots in favour of landmark-based word recognition.

Hand detection with YOLOv8n. As introduced in the related-work discussion and dataset description, an initial hand-localisation experiment trained YOLOv8n [19] on the joint EgoHands and Kaggle Hand Dataset corpus described in Section 5.3.3. The input was the merged and deduplicated bounding-box dataset, approximately 5,223 annotated images after stratified train/validation/test splitting. The output was a detector with approximately **98 % mAP@0.5** and **87 % Mean IoU** on the held-out test split. YOLOv8n was selected after comparison with SSDLite-MobileNetV3 and EfficientDet-D0 because it provided lower inference latency at comparable detection accuracy on this corpus.

Letter-level pixel classification. A parallel track trained CNN classifiers on the union of Sign Language MNIST (DS-03) and ASL Alphabet (DS-04). Coverage was limited to 24 of the 26 ASL letters because *J* and *Z* require hand motion and cannot be represented by a single static image. Accuracy on this static-frame classification task was high, but the task was also far simpler than the eventual project goal: predicting one of $\sim$24 hand poses per frame is a different problem from predicting a word from a multi-frame sign.

What the exploration revealed. The bounding-box detector was not the bottleneck, since it worked reliably. The bottleneck was *representation*. Pixel-based recognition has three properties that are problematic at the WLASL scale: (i) it carries appearance signal (skin tone, lighting, background) that is irrelevant to the linguistic content of a sign and that the model has to learn to discount with limited data; (ii) the parameter cost of an image-conditioned sequence model grows quickly, which makes overfitting on a few hundred clips per class very likely; and (iii) the geometry of hand articulation, which is the actual carrier of meaning, is not made explicit to the model.

The pivot to landmark-based word recognition replaced the visual problem with a geometric one: each frame is reduced from a 224×224 RGB tensor to 21 three-dimensional keypoints (63 features), a reduction of approximately $2{,}389\times$ in input dimensionality. The representation discards background and appearance at the source and exposes the joint geometry directly to a temporal model. The remainder of this chapter studies the resulting pipeline.

6.3 Architecture Selection on WLASL30

This section instantiates Phase M2 from Section 6.1. The objective is to select the architecture family that best converts one-hand MediaPipe landmark sequences into word-level ASL predictions under a fixed WLASL30 split. The input is WLASL30, the 30 most frequent WLASL classes, with 242 train, 56 validation and 34 test clips. Each row in Table 6.1 is the best local configuration retained

for that architecture family before the final comparison. The comparison then holds constant the dataset, landmark pipeline, hardware, optimiser, loss, batch size and early-stopping rule defined in Section 6.1. The Transformer baseline is the only exception in optimiser scale, using $d_{\text{model}} = 256$, $\text{nhead} = 8$, four transformer layers, $\text{lr} = 5 \times 10^{-4}$, $\text{wd} = 1 \times 10^{-4}$ and dropout 0.3, as recorded in its experiment manifest.

Table 6.1: Single-split test results of the five locally tuned architectures evaluated on WLASL30 (34 test samples). Source: self-created from experiment outputs. Standard deviations are not available at this stage; the multi-seed validation of the selected configuration is in Section 6.5.

Architecture	**Top-1 (%)**	**Top-5 (%)**	**Notes**
LSTM (landmarks)	41.18	67.65	Bidirectional, hidden size 256, 2 recurrent layers.
GRU (landmarks)	38.24	76.47	Bidirectional, hidden size 256, 2 recurrent layers.
GRU + Enriched features	47.06	79.41	GRU baseline with bone vectors and joint angles.
GRU + Attention	**58.82**	**91.18**	Selected configuration, 2.26 M parameters.
Transformer	50.00	73.53	Four encoder layers, 8 heads, $d_{\text{model}} = 256$.

The ordering in Table 6.1 is consistent with a low-data setting. Plain GRU and plain LSTM architectures receive only 242 training examples spread over 30 classes, which limits the amount of temporal variation that either recurrent architecture can learn. Adding hand-engineered enriched features, namely bone vectors and inter-joint angles, gives the model a 9-point absolute Top-1 increase over the plain GRU baseline. That gain disappears once temporal attention is added, because the attention layer can learn which frames carry the discriminative signal instead of receiving that information through manually designed geometric features.

The Transformer baseline from this comparison is informative because its parameter budget is competitive with the GRU+Attention model. It underperforms on Top-1, 50.00 versus 58.82, and more sharply on Top-5, 73.53 versus 91.18. This result suggests that a recurrent sequential prior is useful when the training set is small. The experiment does not show that Transformer architectures are unsuitable for SLR in general, only that this specific Transformer configuration did not match the selected recurrent architecture on WLASL30.

The best-performing configuration, a bidirectional GRU with hidden size 256, two layers, dropout 0.4 and a Temporal Attention head, is therefore selected as the default for every downstream experiment in the chapter. Its confusion matrix on the WLASL30 test split is shown in Figure 6.1.

The matrix exposes two structural failure modes. The first is class imbalance at evaluation time: several test classes have only one or two samples, so a single error is enough to drop their per-class recall to zero. The second is genuine visual similarity between specific signs: according to the stored experiment outputs, the most common confusions on this split are *candy* $\rightarrow$ *who* and *what* $\rightarrow$ *book*, both at 2 of 3 affected samples, plus single-sample failures on *fine* $\rightarrow$ *help*, *thanksgiving* $\rightarrow$ *black*, and *cool* $\rightarrow$ *computer*.

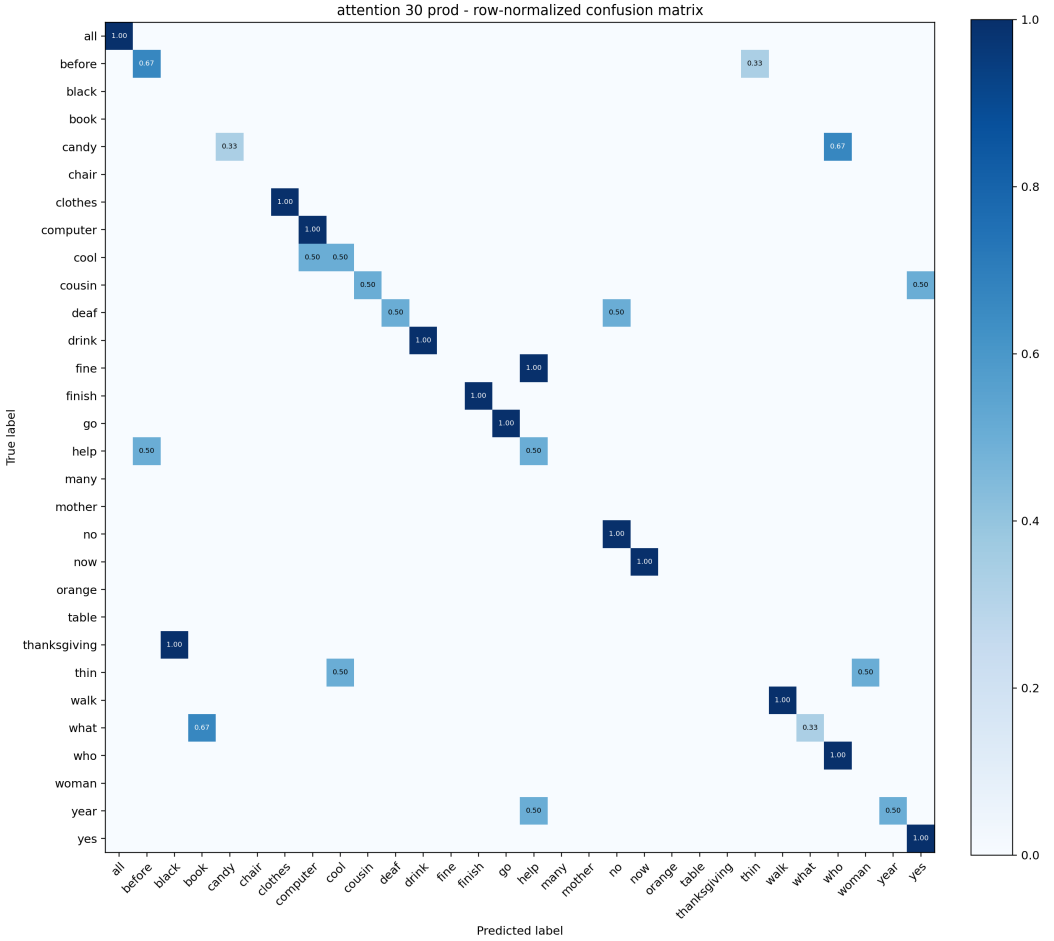


Figure 6.1: Row-normalised confusion matrix of the selected GRU+Attention configuration on the WLASL30 test split (34 samples). Source: self-created from experiment outputs.

The attention layer performs best because sign language clips contain a small number of *key frames* where the linguistic content is concentrated, surrounded by neutral preparation and recovery frames; weighting the GRU outputs by a learned attention distribution focuses capacity where it matters and tolerates variation in execution speed across signers.

6.4 Sequence Length Sweep

This section applies Phase M3 from Section 6.1 to the selected GRU+Attention architecture (Section 6.3). The objective is to determine the maximum sequence length `max_frames`, which is the only hyperparameter that changes the temporal input shape seen by the model. Subsampling too aggressively, as in 32-frame sequences, can discard temporal dynamics. Oversampling, as in 96-frame sequences, can introduce padding and noise on shorter clips. A multi-seed sweep distinguishes the effect of sequence length from initialisation variance.

The sweep covers nine runs: 3 seeds (42, 123, 2026) $\times$ 3 sequence lengths (32, 64, 96), with every other variable held fixed at the values in Section 6.1. Aggregated results from the sweep are reported in Table 6.2.

Table 6.2: Sequence-length sweep on WLASL30 with 3 seeds per row. Mean $\pm$ standard deviation across seeds. Top-1 standard deviations are bounded below by the test-set granularity ($\approx$2.94 pp per misclassification on 34 samples). Source: self-created from experiment outputs.

Frames	**Top-1**	**Top-5**	**Macro-F1**	**Balanced Acc.**
32	56.86 ± 6.12	85.29 ± 2.95	50.52 ± 8.27	64.65 ± 7.48
64	58.82 ± 2.94	78.43 ± 4.49	54.88 ± 0.93	67.42 ± 0.76
96	54.90 ± 4.49	81.37 ± 4.49	49.63 ± 6.33	61.36 ± 6.56

The 64-frame configuration produces the most stable training across the three seeds. Macro-F1 standard deviation drops to 0.93 pp, and Balanced Accuracy standard deviation to 0.76 pp, both an order of magnitude tighter than at 32 or 96 frames. The sweep does not select the configuration that maximises any single mean metric, since 32 frames actually edges out 64 on Top-5 (85.29 vs. 78.43), but it does select the configuration whose mean is least sensitive to the seed. The selection criterion is reproducibility, not peak Top-1, and that is the criterion this chapter applies to every downstream choice.

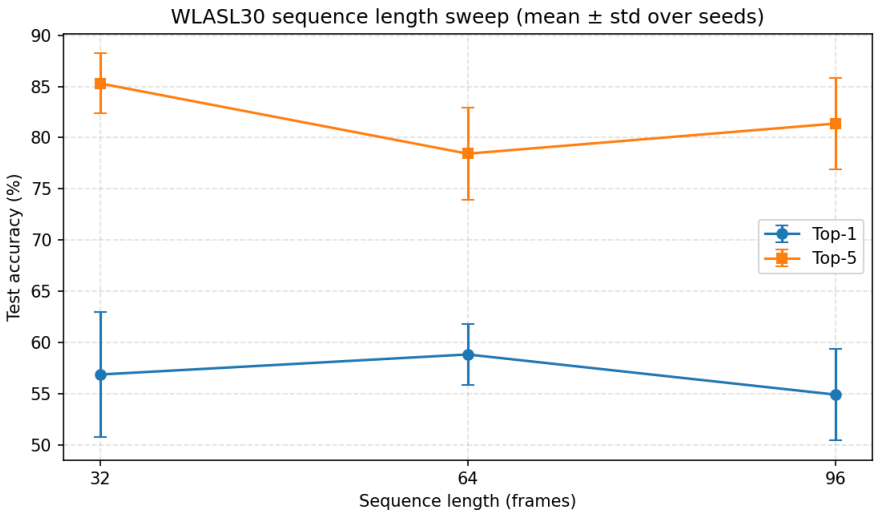


Figure 6.2: Per-metric distribution across the sequence-length sweep (3 seeds at 32, 64, and 96 frames). Source: self-created from experiment outputs.

The 64-frame value is therefore fixed as the default for every downstream experiment in this chapter, including the cross-dataset MS-ASL50 evaluation in Section 6.6. A dedicated sweep on MS-ASL50, where median clip length is approximately 80 frames, would be a natural extension and is acknowledged in Section 6.8.

6.5 WLASL30 Validation

This section applies Phase M4 from Section 6.1. Two independent validation protocols characterise the selected configuration on WLASL30. The first is the **Stratified 5-Fold Cross-Validation** run on WLASL100 filtered to the same 30 classes (332 total samples), which yields a Top-1 mean of **63.85 % ± 4.50 %** and a Top-5 mean of **86.46 % ± 2.04 %**. This is the headline result because it averages across both initialisation and split, and because the standard deviations are computed across non-overlapping test partitions. The second is the **multi-seed evaluation** on the fixed WLASL30 split at $max_frames = 64$ extracted from the sequence-length sweep (Table 6.3).

Table 6.3: Per-seed test results of the selected GRU+Attention configuration on the fixed WLASL30 split (34 test samples). Source: self-created from experiment outputs.

Seed	Top-1	Top-5	Macro-F1	Balanced Acc.	Best Epoch
42	55.88	73.53	55.94	68.18	40
123	61.76	79.41	54.20	66.67	29
2026	58.82	82.35	54.49	67.42	27
Mean ± Std	58.82 ± 2.94	78.43 ± 4.49	54.88 ± 0.93	67.42 ± 0.76	N/A

The two protocols disagree by approximately five percentage points on Top-1 (63.85 vs. 58.82). The disagreement is informative rather than contradictory: 5-Fold CV varies the test partition itself, so the Top-1 average is taken over five non-overlapping test partitions of approximately 66 samples each ($332/5 = 66.4$) and effectively smooths the worst single split; the multi-seed protocol fixes the 34-sample test split and varies only the model initialisation and batch order. Both quantities are real; they answer different questions. The headline number reported elsewhere in the work is the 5-Fold mean because it is the more statistically stable of the two and is directly comparable to the K-Fold standard in the SLR literature.

The per-class behaviour of the selected configuration on the fixed split is concentrated on a small number of low-support classes. According to the per-class metric outputs, the worst recall values among supported classes are *fine* (1 sample, recall 0.00), *thanksgiving* (1, 0.00), *thin* (2, 0.00), *candy* (3, 0.33), *what* (3, 0.33), and *cool* (2, 0.50). With one to three test samples per class, even a single misprediction yields a 33–100 pp recall drop. This is a property of the test split, not of the model: the per-class metric can be recovered only by enlarging the evaluation set, which is exactly what the cross-dataset experiment in Section 6.6 achieves.

6.6 Cross-Dataset Validation: MS-ASL50

This section applies Phase M5 from Section 6.1. The pipeline was tuned on WLASL30. To check whether its behaviour is a property of the method or of the benchmark, the same code path was retrained on MS-ASL50 with no architectural changes and no dataset-specific hyperparameter tuning. The only fields that differ between the two configurations are the dataset path and the gloss vocabulary.

6.6.1 Building the MS-ASL50 subset

MS-ASL ships only annotation JSONs and YouTube URLs; the 25,513 declared instances must be retrieved per-clip, and not all of them are still online. The dataset reconstruction audit records that **16,816 of 25,513 clips were successfully downloaded (65.9 %)**, with split-level coverage of 74.0 % / 47.5 % / 58.0 % (train / val / test). The drop-out is dominated by YouTube videos that have been removed or made private since the dataset was published.

The subset selection itself is constrained by data deficiency. The disk-level audit reports that only **6** of the top-30 classes meet the strict 30 train / 5 val / 5 test threshold; **17** meet the 20 / 5 / 5 threshold; and **57** meet the relaxed 20 / 3 / 3 threshold. An MS-ASL30 reproduction is therefore not viable from the locally available data. The chosen alternative is **MS-ASL50**, defined as the 50 classes with the largest train counts among those satisfying the 20 / 3 / 3 threshold. After landmark extraction, 31 source clips are dropped because their mean MediaPipe detection rate falls below the 0.3 floor, leaving **1,410 train / 306 val / 236 test (1,952 total)**. The mean detection rate across surviving clips is **83.9 %**, substantially above the WLASL figure (approximately 69 %), consistent with MS-ASL clips being longer and shot under more controlled conditions.

6.6.2 Cross-dataset results

Both datasets were trained with three seeds (42, 123, 2026). Aggregated metrics are in Table 6.4, with the MS-ASL50 per-seed breakdown in Table 6.5.

Table 6.4: Side-by-side comparison of the GRU+Attention pipeline on WLASL30 and MS-ASL50. Both rows are aggregated over 3 seeds. Source: self-created from experiment outputs.

Dataset	Classes	Train / Val / Test	Top-1	Top-5	Macro-F1
WLASL30	30	242 / 56 / 34	58.82 ± 2.94	78.43 ± 4.49	54.88 ± 0.93
MS-ASL50	50	1,410 / 306 / 236	70.20 ± 2.89	90.54 ± 0.89	68.08 ± 3.44

Table 6.5: Per-seed test results on MS-ASL50 (236 test samples). Source: self-created from experiment outputs.

Seed	Top-1	Top-5	Macro-F1	Balanced Acc.	Best Epoch
42	66.95	89.83	64.22	66.34	55
123	72.46	90.25	70.81	72.75	120
2026	71.19	91.53	69.21	71.58	92
Mean $\pm$ Std	70.20 ± 2.89	90.54 ± 0.89	68.08 ± 3.44	70.22 ± 3.41	N/A

The central observation is that the Top-1 standard deviations on the two datasets are nearly identical (2.94 pp on WLASL30 versus 2.89 pp on MS-ASL50). With $7\times$ fewer test samples on WLASL30, this is the strongest statement the experiment supports: *training stability is a property of the pipeline, not of a specific benchmark*. The pipeline does not need dataset-specific tuning to produce reproducible results on a second ASL corpus.

The absolute Top-1 figure on MS-ASL50 (70.20) is higher than on WLASL30 (58.82), but this is not evidence that MS-ASL is intrinsically easier. Three structural factors account for the gap. First, MS-ASL50 has approximately $6\times$ more training samples (1,410 vs. 242), which is the dominant variable in this data regime. Second, the test set is $7\times$ larger (236 vs. 34), so each individual misclassification moves Top-1 by approximately 0.42 pp instead of 2.94 pp; a finer measurement grid yields more representative values, which usually means closer to the true asymptotic performance. Third, MS-ASL clips are sourced from longer, better-lit YouTube educational material, which is reflected in the higher MediaPipe detection rate. The experiment therefore supports the claim that the pipeline reproduces; it does not support a claim about intrinsic dataset difficulty.

The full 50-class confusion matrix for the seed-2026 MS-ASL50 run is too dense to read at inline chapter size, so it is provided as a full-page figure in Appendix A, Figure A.1. The confusion pattern, as verified by the stored experiment outputs, is dominated by visually adjacent signs. The largest absolute confusion is *deaf* $\rightarrow$ *need* (4 of 14 samples, 28.6 %), followed by *pencil* $\rightarrow$ *write* and *fish* $\rightarrow$ *lost* (each 3 of 5, 60.0 %), *nurse* $\rightarrow$ *sit* (3 of 6, 50.0 %), and *mother* $\rightarrow$ *woman* (2 of 4, 50.0 %). These are recognisable as one-handed, location-similar pairs whose discriminative cue is fine-grained finger shape rather than gross trajectory.

The per-class metrics show three classes with 0.00 recall (*here*, *man*, *woman*, all with three test samples), and six further classes below 50 % recall (*need* at 0.25, *school* at 0.29, *nurse* at 0.33, *lost* at 0.33, *write* at 0.40, *doctor* at 0.43). At the other end, nineteen classes reach perfect recall on this split, including six with five or more support samples (*teacher* 9, *no* 7, *tired* 7, *deaf* 6, *please* 5,

like 5). The headline Macro-F1 of 68.08 is therefore an average over a strongly bimodal per-class distribution, which is consistent with the higher Macro-F1 standard deviation (3.44) compared to Top-1 (2.89): per-class recall is a finer-grained signal and amplifies seed-to-seed variation in which classes happen to be learned best.

6.7 Vocabulary Scaling

This section applies Phase M6 from Section 6.1 to the selected GRU+Attention configuration. The objective is to measure how recognition performance changes as the WLASL vocabulary grows from 30 classes to 1,500 classes. The input is a sequence of WLASL subsets with progressively lower mean videos per class. The configuration keeps the same landmark representation, recurrent architecture, `max_frames`=64 and seed 42. Multi-seed validation was not performed beyond 30 classes, so the 30-class row is the multi-seed reference and every larger-vocabulary row is a single-seed run.

The output of this phase is a scaling table, a Top-1 curve and a parallel-coordinates diagnostic. Table 6.6 reports the split sizes, test metrics, best epoch and training time for each subset. Figure 6.3 shows the main performance curve. The denser parallel-coordinates view is provided at full-page size in Appendix A, Figure A.2, because its five axes are not legible at inline chapter size.

Table 6.6: Vocabulary scaling of the selected GRU+Attention configuration on WLASL subsets. The 30-class row is the multi-seed mean from Table 6.4; all other rows are single-seed runs and standard deviations cannot be reported. *Train time* is the total wall-clock time of the training run on the NVIDIA RTX 3050 Ti Laptop GPU described in Section 6.1. Source: self-created from experiment manifests.

Vocabulary	Train / Val / Test	Top-1	Top-5	Best Epoch	Train time (s)
30	242 / 56 / 34	58.82 ± 2.94	78.43 ± 4.49	27–40	N/A
100	795 / 186 / 132	44.70	70.45	99	83
300	1,974 / 474 / 360	28.61	59.72	112	224
500	2,967 / 723 / 561	23.71	49.55	111	311
700	3,879 / 974 / 736	19.57	42.66	57	248
900	4,707 / 1,202 / 880	12.50	33.75	220	881
1,000	5,122 / 1,329 / 938	11.19	31.24	113	602
1,200	5,840 / 1,536 / 1,051	10.56	27.69	220	1,105
1,500	6,864 / 1,834 / 1,229	9.76	28.23	220	1,308

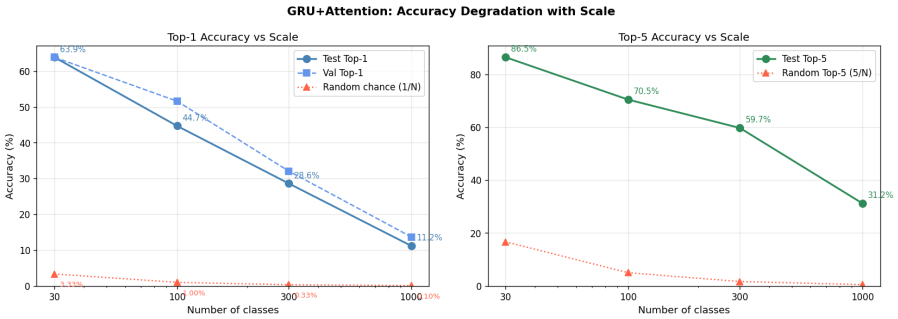


Figure 6.3: Top-1 accuracy as a function of vocabulary size on WLASL. Source: self-created from experiment outputs.

The relationship between vocabulary size and Top-1 accuracy is monotonically decreasing across the entire range and roughly linear in $\log(\text{num_classes})$ between 30 and 1,000 classes. The curve does not show an abrupt implementation failure. Instead, it matches the reduction in per-class data as the vocabulary expands. WLASL2000 has a median of approximately ten samples per class, which is below the regime where a discriminative neural sequence model can separate fine-grained gestural categories without external pretraining or stronger augmentation. The bottleneck is the per-class data budget, not an isolated failure of the architecture.

A weaker secondary observation, restricted to the runs above 700 classes, is that the training curves of the larger subsets show greater sensitivity to early-stopping triggers; runs at 900, 1,200, and 1,500 classes saturate the maximum-epoch budget without an unambiguous improvement, while runs below 700 classes consistently early-stop earlier. This is the practical edge of the data-starvation regime: not a discrete failure point, but a regime where additional training time stops paying back.

The data-starvation interpretation is corroborated by the parallel-coordinates view in Figure A.2, which projects every scaling run onto five normalised axes: mean videos per class, final train accuracy, validation Top-1, test Top-1 and test Top-5. At 30 classes the line stays high across all axes, with a mean of 11.07 videos per class, train accuracy of $99.00\,\%$ and validation Top-1 of $63.85\,\%$. At $1{,}500$ classes, the same relationship collapses from $74.77\,\%$ train accuracy to $11.67\,\%$ validation Top-1, with only 4.58 videos per class. The train-side values remain comparatively high while validation and test metrics fall, which supports the interpretation that sample scarcity drives the scaling behaviour.

6.8 Limitations

These experimental results provide empirical support for several core assumptions, yet they also challenge several prevailing premises. The supported assumptions are that landmark-based input is a viable representation for isolated ASL recognition, that temporal attention improves the recurrent baseline on WLASL30, that the selected pipeline can reproduce on MS-ASL50 without dataset-specific architecture changes and that vocabulary growth is mainly constrained by per-class data availability. At the same time, the results challenge the assumption that Transformer architectures dominate recurrent baselines across all data regimes and the assumption that single-seed reporting is sufficient for small test sets. The remaining constraints are classified into six categories and

addressed systematically below.

Table 6.7: Limitation categories, ordered from data acquisition constraints to modelling choices. Source: self-created from the experimental audit.

Category	Where it appears	Effect on interpretation
Landmark extraction quality	WLASL and MS-ASL landmark preprocessing	Low MediaPipe detection rates reduce the number of informative frames.
Evaluation resolution	WLASL30 fixed test split	Small test sets make single errors change Top-1 by several percentage points.
Dataset reconstruction	MS-ASL download audit	YouTube attrition means the reconstructed subset is time-dependent.
Seed coverage	Vocabulary scaling beyond 30 classes	Single-seed runs show the trend but not the exact seed variance.
Architecture coverage	Cross-dataset validation	Only the selected GRU+Attention configuration was retrained on MS-ASL50.
Training policy	Augmentation and class-imbalance handling	Fixed augmentation and unweighted loss leave minority-class effects unresolved.

Landmark extraction quality. The MediaPipe detection rate on WLASL is approximately $69\,\%$ at the frame level. Clips with motion blur, partially occluded hands or hands moving outside the frame produce silent failures that the linear-interpolation step mitigates but does not erase. The interpolated frames carry no new information, so any clip with sustained low detection rate effectively provides fewer informative frames than its nominal length suggests. The same caveat applies less strongly to MS-ASL50, where the average detection rate is $83.9\,\%$.

Evaluation resolution. The WLASL30 test split is small, with 34 samples. Even at three seeds, the standard deviation in Top-1 is bounded below by $\approx$2.94 pp because that is the resolution of one misclassification. The 58.82 ± 2.94 figure should be read as an interval of expected runs, not as a precise estimate of true generalisation accuracy. The 5-Fold CV mean of 63.85 partially mitigates this by averaging over five different test partitions, although the absolute resolution per fold remains limited.

Dataset reconstruction. YouTube attrition on MS-ASL is $34.1\,\%$ of source clips. The MS-ASL50 subset reproducible from the current YouTube state is therefore not identical to the one that was reproducible when the dataset was released. Any direct numerical comparison with results published against the original MS-ASL release [2] must account for this difference in available clips.

Seed coverage. Beyond 30 classes, every reported result is single-seed. Differences between adjacent vocabulary sizes in Section 6.7, for example 23.71 at 500 classes versus 19.57 at 700 classes, may sit within plausible seed-to-seed variation. The downward trend is supported by all scaling runs, but the exact slope should not be treated as a multi-seed estimate.

Architecture coverage. Only the selected GRU+Attention configuration was evaluated cross-dataset. A run of the specific Transformer baseline from Table 6.1 on MS-ASL50 would test whether its underperformance in Section 6.3 reflects the WLASL30 low-data setting or the architecture choice itself. That experiment is a natural extension, but it was not executed within the scope of this project.

Training policy. The augmentation pipeline was kept fixed throughout the chapter: rotation, scaling, translation, Gaussian noise and horizontal flip. Horizontal flip conflates left-handed and right-handed signers, which is acceptable for a small-vocabulary one-hand pipeline but would need re-examination for two-hand signs and for downstream studies of signer-handedness bias. Class-weighted loss and minority-class oversampling were also not applied, despite the per-class disparities documented in Sections 6.5 and 6.6.

Part II: Qualitative Evaluation

6.9 Qualitative Validation

The quantitative experiments in Part I evaluate recognition accuracy under controlled benchmark conditions. This section evaluates a different question: whether the SignToText prototype is understandable, responsive and useful when users interact with the application. The questionnaire therefore measures usability and perceived value, not the technical accuracy of the model, which was validated through the benchmark experiments above.

6.9.1 Survey Design

The qualitative validation used a short Google Form completed after participants tested the SignToText demo. The form introduced the application as an ASL recognition prototype that translates detected signs into text and speech. It then collected three types of data: previous knowledge of sign language, Likert-scale usability ratings and open-ended feedback about confusion points or requested changes. The expected completion time was approximately five minutes.

The Likert block used a 1–5 scale and evaluated five dimensions: first impression, understood as whether the participant quickly knew what the app did and how to start using it, clarity of the buttons and capture modes, comprehensibility of the prediction results, perceived response speed and perceived utility. The feature-preference question allowed multiple selections, including automatic text-to-speech, Top-K alternatives with confidence percentages, temporal attention maps, model selection, continuous phrase mode, countdown before recording, gallery upload and processing statistics. The recommendation question used a 1–10 scale and was reduced to Net Promoter Score (NPS), computed as the percentage of promoters minus the percentage of detractors.

6.9.2 Results

The questionnaire collected responses from $N = 15$ participants. Table 6.8 summarises the Likert dimensions. The strongest dimension is perceived utility, with a mean of 4.5 out of 5, followed by perceived speed at 4.3 and first impression at 4.2. The lowest dimension is result comprehension, with a mean of 3.7, which indicates that the presentation of the prediction output still needs refinement.

Table 6.8: Usability questionnaire results for the SignToText demo. Means are computed on a 1–5 Likert scale. Source: self-created from Google Form responses.

Dimension	Mean score
First impression	4.2
Button and capture-mode clarity	4.0
Result comprehension	3.7
Perceived speed	4.3
Perceived utility	4.5

The recommendation item produced an NPS of $+40$. This is a positive usability signal for a research prototype, since more participants were willing to recommend the application than reject it. The result should not be interpreted as market validation because the sample is small and informal, but it does indicate that the interaction concept was received favourably.

Table 6.9 reports the three most valued features. Top-K alternatives with confidence percentages were selected by 87 % of participants, making them the most valued interface element. Automatic text-to-speech was selected by 73 %, and model selection by 60 %. These results support the design choice of exposing a prediction shortlist rather than presenting only the Top-1 label. Other evaluated features, including the temporal attention map, continuous phrase mode, countdown before recording, gallery upload and processing statistics, received markedly fewer selections.

Table 6.9: Most valued SignToText features according to the usability questionnaire. Participants could select multiple options. Source: self-created from Google Form responses.

Feature	Participants selecting it
Top-K alternatives with confidence percentages	87 %
Automatic text-to-speech	73 %
Selection between several trained models	60 %

6.9.3 Discussion

The qualitative results show that users understood the prototype's purpose and perceived it as useful. The high perceived-speed score also suggests that the end-to-end interaction did not feel slow during the demo.

The main friction appears in result comprehension. Free-text responses and observed user reactions showed that the temporal attention map and processing statistics confused several participants when shown without context. Prediction alternatives and speech output are useful to users, while diagnostic visualisations should be hidden behind an advanced-details control or accompanied by short explanatory labels.

Chapter 7

Ethical Implications and Social Impact

7.1 Ethical Impact of AI-Based Systems

An ethical analysis in AI examines the implications, concerns and challenges associated with the development and deployment of these systems. As AI is applied to increasingly sensitive contexts, technical evaluation must be paired with an explicit ethics review that assesses effects on users, data subjects and wider social contexts. This introductory review uses four technical dimensions commonly identified across AI ethics frameworks: **Human Rights and Fairness** (preventing bias and discrimination), **Accountability** (clarifying who is responsible for the system's decisions), **Privacy and Security** (protecting user data and confidentiality) and **Transparency** (avoiding opaque black-box behaviour). These dimensions are operationalised in Section 7.2 through the seven principles of Google's AI framework, which is the specific framework chosen for the project evaluation.

7.2 Ethical Evaluation Framework: Google's AI Principles

The framework **chosen** for this evaluation is **Google's AI Principles** [32], originally published in 2018 and refined annually; the primary reference for this analysis is the **2023 Progress Update** [33]. The framework was chosen because its seven principles are formulated as concrete objectives that map cleanly onto the design decisions of a research project. Google's annual progress reports demonstrate how the principles apply to systems that share characteristics with this work (computer vision and on-device inference). The seven principles are:

1. **Be socially beneficial:** address a real societal need and do so effectively.
2. **Avoid creating or reinforcing unfair bias:** ensure fair and inclusive treatment.
3. **Be built and tested for safety:** prioritise safety in design, validation and operation.
4. **Be accountable to people:** provide transparency and user control.
5. **Incorporate privacy design principles:** protect user data by design.

6. **Uphold high standards of scientific excellence:** follow rigorous methodology.
7. **Be made available for uses that accord with these principles:** prevent misuse and align deployment with the framework.

7.3 Project Analysis Against Google's AI Principles

This section evaluates the project against each principle using scored metrics. Each identifier uses the format Mx.y, where x denotes the principle number and y denotes the metric within that principle. Each metric uses a three-point scale: 0 (not addressed), 5 (partially addressed) and 10 (fully addressed). Averages are reported to one decimal place for consistency across principles. Some metrics derive from experimental results, whereas others reflect design decisions or documentation practices. Table 7.1 consolidates the scores, per-principle averages and global score.

7.3.1 Principle 1: Be Socially Beneficial

Sign language is the primary means of communication for millions of deaf and hard-of-hearing persons, yet significant communication barriers still exist in interactions with hearing people. An automatic American Sign Language (ASL) recognition system can act as a bridge in educational, medical and everyday contexts.

- **M1.1: Real societal need.** The World Health Organization estimates over 430 million people worldwide have disabling hearing loss; the project targets a clear accessibility problem. *Score: 10.*
- **M1.2: Effectiveness of the solution.** As summarised in Table 7.1, the best-performing configuration achieves **63.85% Top-1 / 86.46% Top-5 (5-fold cross-validation, WLASL30)**. The same configuration is reproduced under the modular pipeline as 58.82% $\pm$ 2.94% Top-1 across three seeds and extends to **70.20% $\pm$ 2.89% Top-1 / 90.54% $\pm$ 0.89% Top-5** on MS-ASL50. Accuracy degrades at scale (44.70% at 100 classes, 28.61% at 300, 11.19% at 1000), so the system remains a research prototype rather than a deployment-ready translator. This limitation supports an intermediate effectiveness assessment. *Score: 5.*
- **M1.3: Accessibility of the technology.** Built on open-source tools (Python, PyTorch, MediaPipe) and public datasets (WLASL, MS-ASL), with the methodology documented in this work. The recognition pipeline was designed with mobile-first deployment in mind. *Score: 10.*

7.3.2 Principle 2: Avoid Creating or Reinforcing Unfair Bias

Sign language recognition is particularly bias-sensitive: signers vary in hand size, skin tone, signing speed and style. Bias arises both at the perception stage (hand detection and landmark extraction operating on raw images) and at the recognition stage.

- **M2.1: Dataset diversity (word recognition).** WLASL aggregates videos from multiple online sources, providing implicit signer variation, but lacks demographic annotations (skin tone, age, hand morphology), so diversity cannot be quantified formally. *Score: 5.*
- **M2.2: Dataset diversity (hand detection and letter classification).** The auxiliary datasets were:

1. EgoHands (∼4,800 egocentric images) for hand detection.
2. The Kaggle Hand Dataset (∼4,200 third-person images) for hand detection.
3. Sign Language MNIST (34,627 grayscale images for letter classification, with MNIST referring to the Modified National Institute of Standards and Technology dataset format).
4. The ASL Alphabet Dataset (∼87,000 RGB images) for letter classification.

None of these datasets provides demographic metadata, so potential biases cannot be ruled out. *Score: 5.*

- **M2.3: Input representation fairness.** The word-recognition model consumes MediaPipe landmarks (21 keypoints $\times$ 3 coordinates) instead of raw pixels. Operating on geometric joint positions rather than visual appearance reduces sensitivity to skin colour, lighting and background, mitigating appearance-based bias at the recognition stage (though not at the upstream landmark extractor). *Score: 10.*
- **M2.4: Per-class performance analysis.** Per-class metrics were logged across all experiments, surfacing wide accuracy gaps (e.g. "what" 8.3%, "cool" 12.5%, vs. "orange" 100%, "table" 100%). The disparities are mostly explained by visual similarity between signs and small per-class sample counts rather than by demographic factors. Class-weighted loss or oversampling were not applied in this iteration but are facilitated by the existing per-class logging. *Score: 5.*

7.3.3 Principle 3: Be Built and Tested for Safety

In this context, safety means producing reliable predictions and avoiding misleading outputs that could cause misunderstandings in sensitive domains.

- **M3.1: Cross-validation methodology.** Evaluation uses Stratified 5-Fold Cross-Validation (mean Top-1 **63.85% $\pm$ 4.50%**, mean Top-5 **86.46% $\pm$ 2.04%**), corroborated by a 3-seed multi-seed sweep on the same dataset (58.82% $\pm$ 2.94% Top-1) under the refactored pipeline. Test-Time Augmentation was deliberately avoided because it inflates reported accuracy without reflecting true generalisation. *Score: 10.*
- **M3.2: Scalability testing.** The model was retrained on progressively larger WLASL subsets (30 / 100 / 300 / 500 / 700 / 900 / 1000 / 1200 / 1500 classes), documenting accuracy degradation and an instability point above 700 classes. This stress-test establishes the boundaries within which the system can be considered reliable. *Score: 10.*
- **M3.3: Hand detection reliability.** The YOLOv8n hand detector reached approximately 98% mAP50 on the combined test set, validated across both source datasets. A reliable detector is a prerequisite for the recognition pipeline. *Score: 10.*
- **M3.4: Failure mode documentation.** Per-class confusion matrices identify systematically confused word pairs, but a confidence-threshold mechanism (so the system can abstain when uncertain) is not yet implemented and is needed before any production deployment. *Score: 5.*

7.3.4 Principle 4: Be Accountable to People

Accountability requires that the system is interpretable to its users and gives them meaningful control over its outputs.

- **M4.1: Model interpretability.** The Temporal Attention layer produces per-frame weights that can be inspected to understand which frames drove a prediction. This provides a degree of post-hoc interpretability that fully opaque architectures lack. *Score: 10.*
- **M4.2: Top-K predictions.** The system exposes Top-5 alongside Top-1. With Top-5 accuracy of 86.46% on WLASL30 (and 90.54% on MS-ASL50), the correct answer is almost always in the shortlist, allowing the user to make the final call rather than accepting a single forced prediction. *Score: 10.*
- **M4.3: Reproducibility.** All experiments are driven by versioned YAML configs and produce manifests with git commit, hyperparameters and metrics. The main caveat is that some WLASL videos rely on YouTube URLs that may decay over time; this is also an ethical concern about data provenance, since the original signers did not explicitly consent to use of their videos for machine learning research. *Score: 5.*

7.3.5 Principle 5: Incorporate Privacy Design Principles

Privacy applies to three distinct stages of the project: training the hand detector on real photographs, training the word recognition model on landmark abstractions and serving inference from a mobile client.

- **M5.1: Data minimisation through landmarks (training).** Each video frame is reduced to 63 numerical values that discard the original visual content. Once landmarks are extracted, the original video is no longer needed and the processed data cannot reconstruct the signer's face, body or background. *Score: 10.*
- **M5.2: Privacy in the hand detection phase.** Detector training relied on real photographs of identifiable persons (EgoHands, Kaggle Hand). Although these datasets are publicly available for research, no independent privacy audit was conducted on their provenance and the images were used as-is during training. *Score: 5.*
- **M5.3: Privacy in the inference pipeline.** The current FastAPI server receives full RGB frames (416×416) from the mobile app, processes them and persists raw frames and cropped regions to disk without encryption or automatic expiration. This is the weakest privacy point. A natural mitigation is to move landmark extraction to the client device so that only the abstract 63-dimensional vector ever leaves the phone. *Score: 0.*
- **M5.4: K-anonymity considerations.** WLASL stores no identifying metadata about signers. The landmark representation acts as a generalisation: different persons performing the same sign yield similar feature vectors [34]. A formal k-anonymity algorithm was not applied, but the pipeline achieves a comparable effect for the recognition stage. The protection does not extend to the detector training data nor to the server inference path. *Score: 5.*

7.3.6 Principle 6: Uphold High Standards of Scientific Excellence

- **M6.1: Systematic model comparison (word recognition).** Multiple architectures were trained under matched conditions: CNN+LSTM (pixel-based), LSTM (landmark-based), plain GRU, GRU with enriched features, GRU + Temporal Attention and a Transformer variant. *Score: 10.*

- **M6.2: Systematic model comparison (hand detection).** The hand detection stage compared YOLOv8n, SSDLite-MobileNetV3 and EfficientDet-D0 on the same combined dataset using standard detection metrics (mAP50, mAP50-95 and latency). *Score: 10.*
- **M6.3: Statistical validation.** Stratified 5-Fold Cross-Validation reports mean and standard deviation across folds (63.85% $\pm$ 4.50% Top-1). The modular pipeline contributes additional multi-seed runs on WLASL30 and MS-ASL50 to bound model-init variance. Pearson correlation analysis quantifies the relationship between vocabulary size and accuracy. *Score: 10.*
- **M6.4: Literature grounding.** The State of the Art chapter situates each architectural choice (CNN, RNN, LSTM, GRU, Attention, MediaPipe) in the established literature. *Score: 10.*
- **M6.5: Honest reporting of limitations.** The project explicitly documents accuracy degradation at scale, the $\sim$69% MediaPipe detection rate on WLASL and the instability observed above 700 classes. Negative results are reported alongside positive findings. *Score: 10.*

7.3.7 Principle 7: Be Made Available for Uses That Accord with These Principles

- **M7.1: Intended use clarity.** The project frames the system as a research prototype for ASL word recognition, with assistive technology as the target use case. Scope and limitations are explicit. *Score: 10.*
- **M7.2: Misuse potential assessment.** Landmark vectors carry very limited misuse potential (no surveillance or facial recognition is recoverable from 63 floats per frame). The detector and the inference server, however, do process raw images and currently lack access controls and retention policies; this gap should be closed before any deployment beyond the research context. *Score: 5.*
- **M7.3: Open research contribution.** The methodology and code are documented in the project report and in the repository. Trained models and processed datasets have not yet been published in a public artefact registry. *Score: 5.*

7.4 Impact Scale of Ethical Aspects

Table 7.1 consolidates all metrics, their individual scores and the per-principle average.

Principle	ID	Metric	Score	Avg.
P1: Socially Beneficial	M1.1	Addresses a real societal need	10	**8.3**
	M1.2	Effectiveness of the solution	5	
	M1.3	Accessibility of the technology	10	
P2: Avoid Unfair Bias	M2.1	Dataset diversity (word recognition)	5	**6.3**
	M2.2	Dataset diversity (detection & letters)	5	
	M2.3	Input representation fairness	10	
	M2.4	Per-class performance analysis	5	
P3: Safety	M3.1	Cross-validation methodology	10	**8.8**
	M3.2	Scalability testing	10	
	M3.3	Hand detection reliability	10	
	M3.4	Failure mode documentation	5	
P4: Accountability	M4.1	Model interpretability	10	**8.3**
	M4.2	Top-K predictions	10	
	M4.3	Reproducibility	5	
P5: Privacy	M5.1	Data minimisation (landmark training)	10	**5.0**
	M5.2	Privacy in detection phase	5	
	M5.3	Privacy in inference pipeline	0	
	M5.4	K-anonymity considerations	5	
P6: Scientific Excellence	M6.1	Model comparison (word recognition)	10	**10**
	M6.2	Model comparison (hand detection)	10	
	M6.3	Statistical validation	10	
	M6.4	Literature grounding	10	
	M6.5	Honest reporting of limitations	10	
P7: Principled Availability	M7.1	Intended use clarity	10	**6.7**
	M7.2	Misuse potential assessment	5	
	M7.3	Open research contribution	5	
		Overall Average Score		**7.6**

Table 7.1: Ethical evaluation metrics based on Google's AI Principles [33]

7.5 Conclusions of the Ethical Analysis

The project attains a composite ethical maturity score of **7.6/10**, reflecting a high standard of technical integrity with specific architectural risks that remain unresolved.

The evaluation identifies scientific excellence (**10/10**) and safety (**8.8/10**) as the strongest dimensions. These scores reflect the rigour of the multi-seed validation strategy, the systematic reporting of limitations and the reliability of the best-performing configuration within the evaluated scope.

The privacy dimension, rated **5.0/10**, represents the primary area for improvement. This deficit is attributed to the inference pipeline, where real-time data processing lacks the same level of protection as the static training environments.

One architectural decision, the use of MediaPipe hand landmarks as input, propagates positive effects across multiple principles: it reduces appearance-based bias (P2), it implements data minimisation that protects identity by construction (P5) and it limits misuse potential because abstract landmarks cannot be repurposed for surveillance (P7). The same advantage, however, does not extend to the upstream stages that operate on raw images. The current FastAPI server receives, processes and persists full RGB frames without encryption or automatic deletion. The hand detector was trained on photographs of identifiable persons. The most actionable mitigation is to relocate landmark extraction to the client device, so that only the 63-dimensional vector ever leaves the phone.

The intermediate scores in bias avoidance (6.3/10) and principled availability (6.7/10) reflect missing formal processes rather than poor design choices: no demographic audit of signers was conducted, the auxiliary detection datasets lack diversity metadata and the server prototype has no access controls or retention policies. Honest reporting of these gaps, including the privacy weakness of the inference pipeline identified in this chapter and the documented accuracy collapse at large vocabularies, is itself an ethical practice aligned with Principle 6 and is a prerequisite for any responsible deployment of the system.

Chapter 8

My Journey at UFV

Chapter 9

Conclusiones

9.1 General Conclusions

9.2 Conclusions regarding Project Objectives

9.3 Next Steps

Bibliography

[1] D. Li, C. Rodriguez, X. Yu, and H. Li, "Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1459–1469.

[2] H. R. V. Joze and O. Koller, "Ms-asl: A large-scale data set and benchmark for understanding american sign language," in *British Machine Vision Conference (BMVC)*, 2019.

[3] S. Ramírez, *Fastapi*, https://fastapi.tiangolo.com/, Accessed: 2026, 2018.

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1097–1105.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[7] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *European Conference on Computer Vision (ECCV)*, Springer, 2014, pp. 818–833.

[8] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "Mediapipe hands: On-device real-time hand tracking," in *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*, 2020.

[9] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994. DOI: 10.1109/72.279181.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *NIPS 2014 Deep Learning Workshop*, 2014.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[13] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.

[14] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[15] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "Ssd: Single shot multibox detector," in *European Conference on Computer Vision (ECCV)*, Springer, 2016, pp. 21–37.

[16] M. Tan, R. Pang, and Q. V. Le, "Efficientdet: Scalable and efficient object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 781–10 790.

[17] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," pp. 779–788, 2016. DOI: 10.1109/CVPR.2016.91.

[18] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[19] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics yolov8*, https://github.com/ultralytics/ultralytics, Accessed: 2025, 2023.

[20] O. Koller, "Quantitative survey of the state of the art in sign language recognition," *arXiv preprint arXiv:2008.09918*, 2020.

[21] R. Rastgoo, K. Kiani, and S. Escalera, "Sign language recognition: A deep survey," vol. 164, Elsevier, 2021, p. 113 794.

[22] D. Bragg, O. Koller, M. Bellard, L. Berke, P. Boudreault, A. Braffort, N. Caselli, M. Huenerfauth, H. Kacorri, T. Verhoef, et al., "Sign language recognition, generation, and translation: An interdisciplinary perspective," pp. 16–31, 2019.

[23] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden, "Sign language transformers: Joint end-to-end sign language recognition and translation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 023–10 033.

[24] K. Yin and J. Read, "Better sign language translation with STMC-transformer," in *Proceedings of the 28th International Conference on Computational Linguistics*, International Committee on Computational Linguistics, 2020, pp. 5975–5989. DOI: 10.18653/v1/2020.coling-main.525.

[25] M. Bohacek and M. Hruz, "Sign pose-based transformer for word-level sign language recognition," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, 2022, pp. 182–191. DOI: 10.1109/WACVW54805.2022.00024.

[26] J. Eunice, Y. Sei, and D. J. Hemanth, "Sign2pose: A pose-based approach for gloss prediction using a transformer model," *Sensors*, vol. 23, no. 5, p. 2853, 2023. DOI: 10.3390/s23052853.

[27] S. Albanie, G. Varol, L. Momeni, T. Afouras, J. S. Chung, N. Fox, and A. Zisserman, "Bsl-1k: Scaling up co-articulated sign language recognition using mouthing cues," in *European Conference on Computer Vision (ECCV)*, Springer, 2020, pp. 35–53. DOI: 10.1007/978-3-030-58621-8_3.

[28] S. Bambach, S. Lee, D. J. Crandall, and C. Yu, "Lending a hand: Detecting hands and recognizing activities in complex egocentric interactions," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1949–1957.

[29] A. Nikkhah, *Hand dataset*, https://www.kaggle.com/datasets/armannikkhah/hand-dataset, Accessed: 2025, 2023.

[30] TechnicianLP, *Sign language mnist dataset*, https://www.kaggle.com/datamunge/sign-language-mnist, Accessed: 2025, 2017.

[31] Akash, *Asl alphabet dataset*, https://www.kaggle.com/grassknoted/asl-alphabet, Accessed: 2025, 2018.

[32] S. Pichai, *Ai at google: Our principles*, https://blog.google/technology/ai/ai-principles/, Published June 7, 2018. Accessed: 2026, 2018.

[33] Google, "Ai principles progress update 2023," Google, Tech. Rep., 2023, 5th edition of Google's annual AI Principles progress report. Accessed: 2026.

[34] L. Sweeney, "K-anonymity: A model for protecting privacy," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, pp. 557–570, 2002. DOI: 10.1142/S0218488502001648.

Appendix A

Additional Experimental Figures

This appendix contains large experimental figures that are required for evaluation but are not legible at the standard inline size used in the main chapters. They are referenced from Chapter 6 where their corresponding results are discussed.

A.1 Large Confusion Matrix

Figure A.1 shows the row-normalised confusion matrix for the MS-ASL50 seed-2026 run. The matrix is placed here because the 50-class layout requires a full page to make the class labels and off-diagonal errors readable.

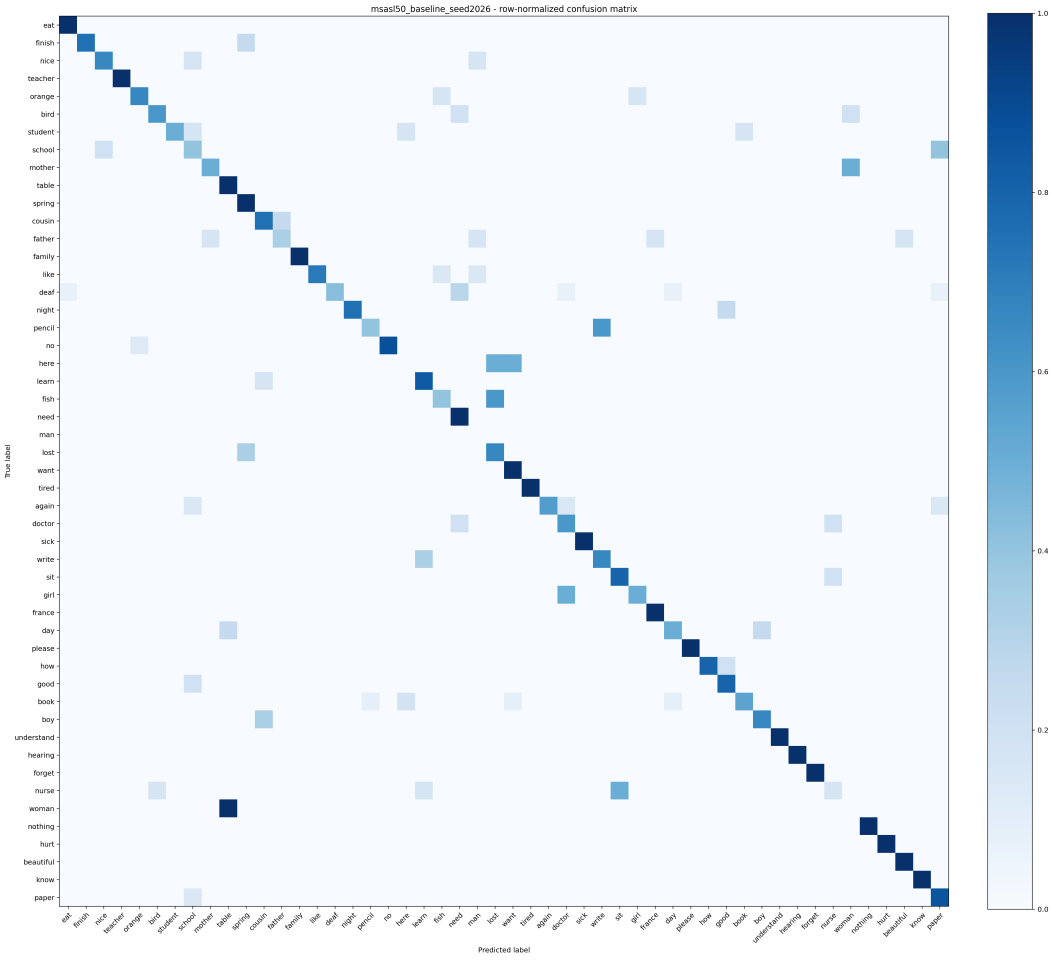


Figure A.1: Row-normalised confusion matrix on the MS-ASL50 test split for seed 2026 (236 samples). Source: self-created from experiment outputs.

A.2 Vocabulary Scaling Diagnostic

Figure A.2 shows the parallel-coordinates diagnostic used in Section 6.7. Each line corresponds to one vocabulary-scaling run, projected onto mean videos per class, final train accuracy, validation Top-1, test Top-1 and test Top-5.

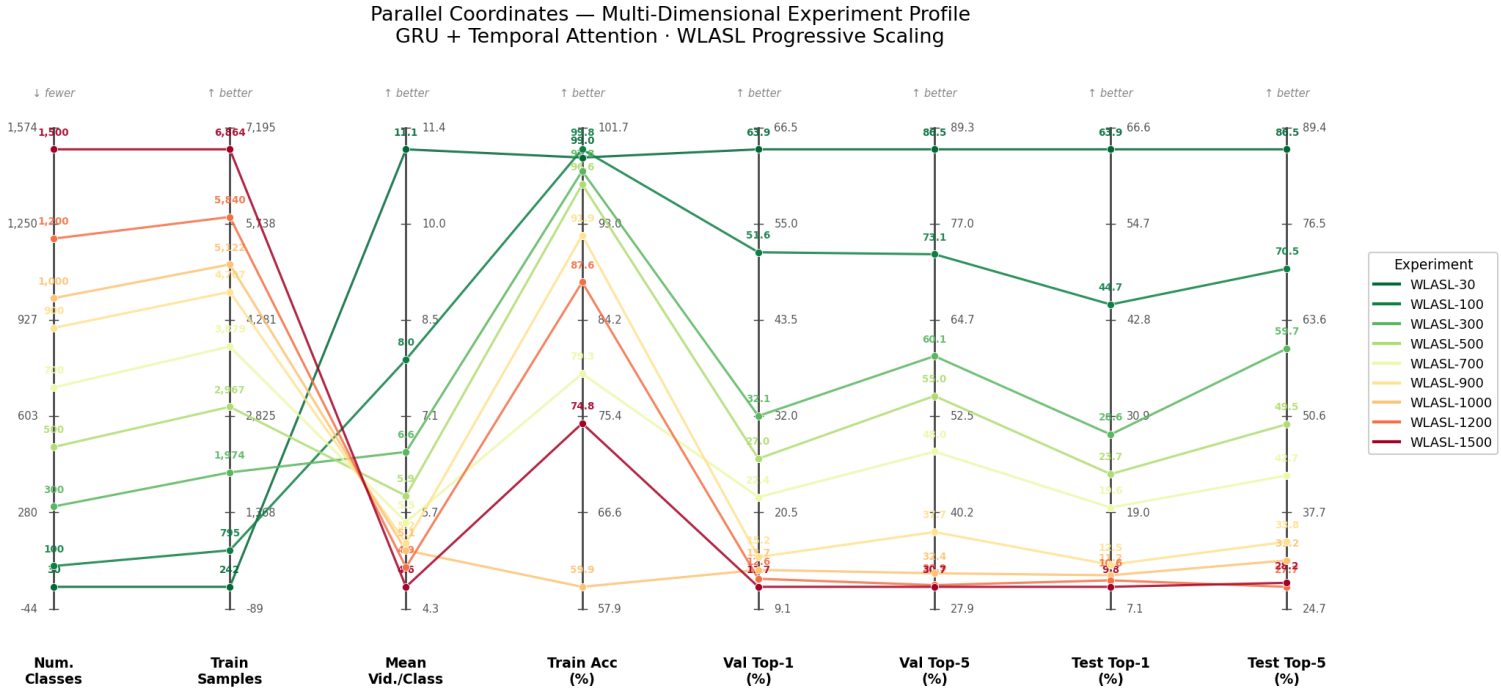


Figure A.2: Parallel-coordinates view of the selected GRU+Attention scaling line on WLASL. Each line is one experiment, coloured from green for smaller vocabularies to red for larger vocabularies. Source: self-created from experiment outputs.